DATAGEEKS · 27 OCTOBER 2026

Stop reranking your way out of a bad representation

Retrieval as an algebra over one textual space, with the model doing the understanding

Edward C. Zimmermann · CoreQuarry · edz@CoreQuarry.com

Edward Zimmermann

Data Scientist/AI/Computer Vision Researcher

https://www.linkedin.com/in/edwardzimmermann

You don’t need a hyper-scaler to search a library.

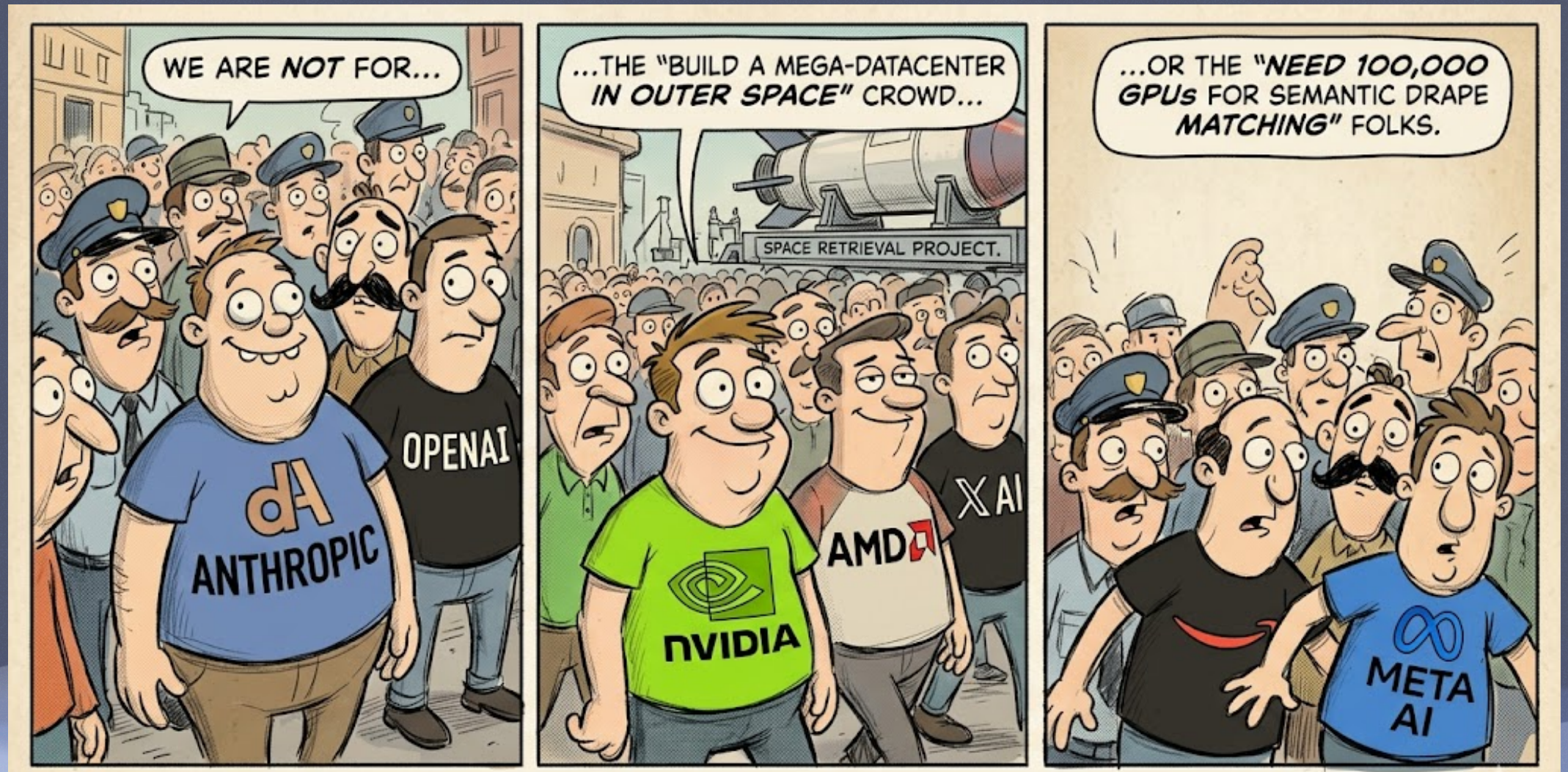
656,519 Patents (XML), 2 billion words, 324 fields, multiple data-types indexed in 48 minutes on a 2021 laptop

Our Local-First Aims

Prioritize data sovereignty, low-latency execution, cost predictability, and absolute user privacy.

Squeeze maximum structural intelligence, deterministic precision, and state-of-the-art neural intent out of local, edge, and consumer hardware (the hardware people have).

100% Open Source (Apache 2.0 licence).



COREQUARRY IS DIFFERENT. Off-Grid. Precise. For Bare Metal.

DATAGEEKS · 27 OCTOBER 2026

IR Beyond Chunks

CoreQuarry · corequarry.com

THE PROBLEM

Four systems to repair one flattened record.

BM25	Embeddings	RRF	Reranker
words, per record	meaning, per chunk	glue the two lists	a model to undo the damage

Each stage exists because the stage before it threw structure away.

THE PROBLEM - OUR CONJECTURE

Retrieval is viewed as one shot. Agents need a conversation.

TODAY

query → top-k chunks → LLM

The model can't say "narrow it to the claims" or "only 1988". It can only ask again, in words.

WHAT AN AGENT NEEDS

search → narrow → widen → follow

With an algebraic language it can compose, with answers that come back places it can go next.

THE PROBLEM

Rewriting the query is a patch on the index.

HyDE	An LLM writes a fake answer so the vector lands nearer the real one.
Query rewrite	An LLM guesses the words the document might have used.
Reranker	A cross-encoder re-reads the candidates it was handed.

Every patch costs a model call, and none of them can reach a part of a record the index never kept track of.

HOW WE MEASURE

The lesson everyone took from BEIR: BM25 beats embeddings.

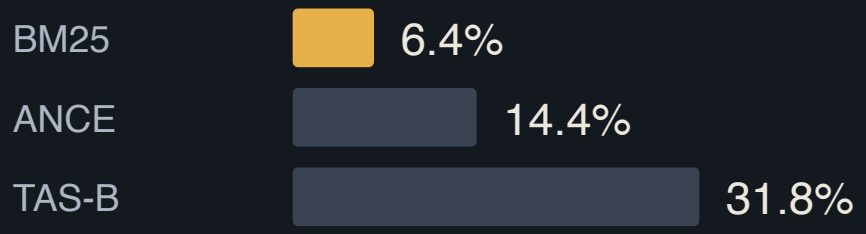
> *“Fine-tuned semantic vector models excel on in-domain data, but their performance drastically degrades in zero-shot environments … the classic lexical baseline (BM25) systematically beat state-of-the-art dense vector models due to semantic drift.”*
>
> The usual retelling of Thakur, Reimers, Rücklé, Srivastava & Gurevych, *BEIR* (2021), arXiv:2104.08663

Repeated until “beats” sounds like a measurement.

HOW WE MEASURE

There is no “beats”.

UNJUDGED RESULTS IN THE TOP 10 (BEIR §6)



Unjudged counts as wrong. In the authors’ words, this can give “an unfair disadvantage to non-lexical approaches.”

The abstract’s winner isn’t BM25.
Re-ranking and late-interaction models score best on average, at high computational cost. BM25 is “a robust baseline”.

Robust is not better.
A tuned model leaves its domain. BM25 never had one to leave.

Words and meaning are different evidence.
Ask what each one tells you, not which one wins.

HOW WE MEASURE

The benchmark rewards the stack it was built around.

Pooled judgments

Only documents some system already returned were ever judged. A method that finds something new is scored as wrong.

Flattened schema

Title and text become one string. Fields, structure and hierarchy are not part of the task.

Encoder-sized documents

Inputs fit a 512-token window, so long records are not what is being tested.

Structure defines meaning

"Nur der Satz hat Sinn; nur im Zusammenhange des Satzes hat ein Name Bedeutung.”

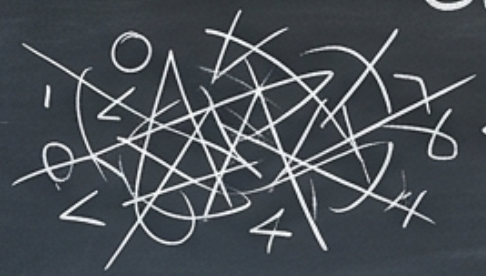
"Only the proposition has sense; only in the context of a proposition has a name meaning."

— *Tractatus Logico-Philosophicus, 3.3*

"Das Satzzeichen besteht darin, daß sich seine Elemente, die Wörter, in ihm auf eine bestimmte Art und Weise zueinander verhalten.” — 3.14.

“"A propositional sign consists in the fact that its elements—the words—stand in a determinate relation to one another in it.” — 3.14

WITHOUT STRUCTURE
CHAOS & DISSOLUTION



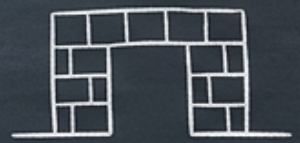
← CHAOS

STABILITY
& MEANING

DISSOLUTION

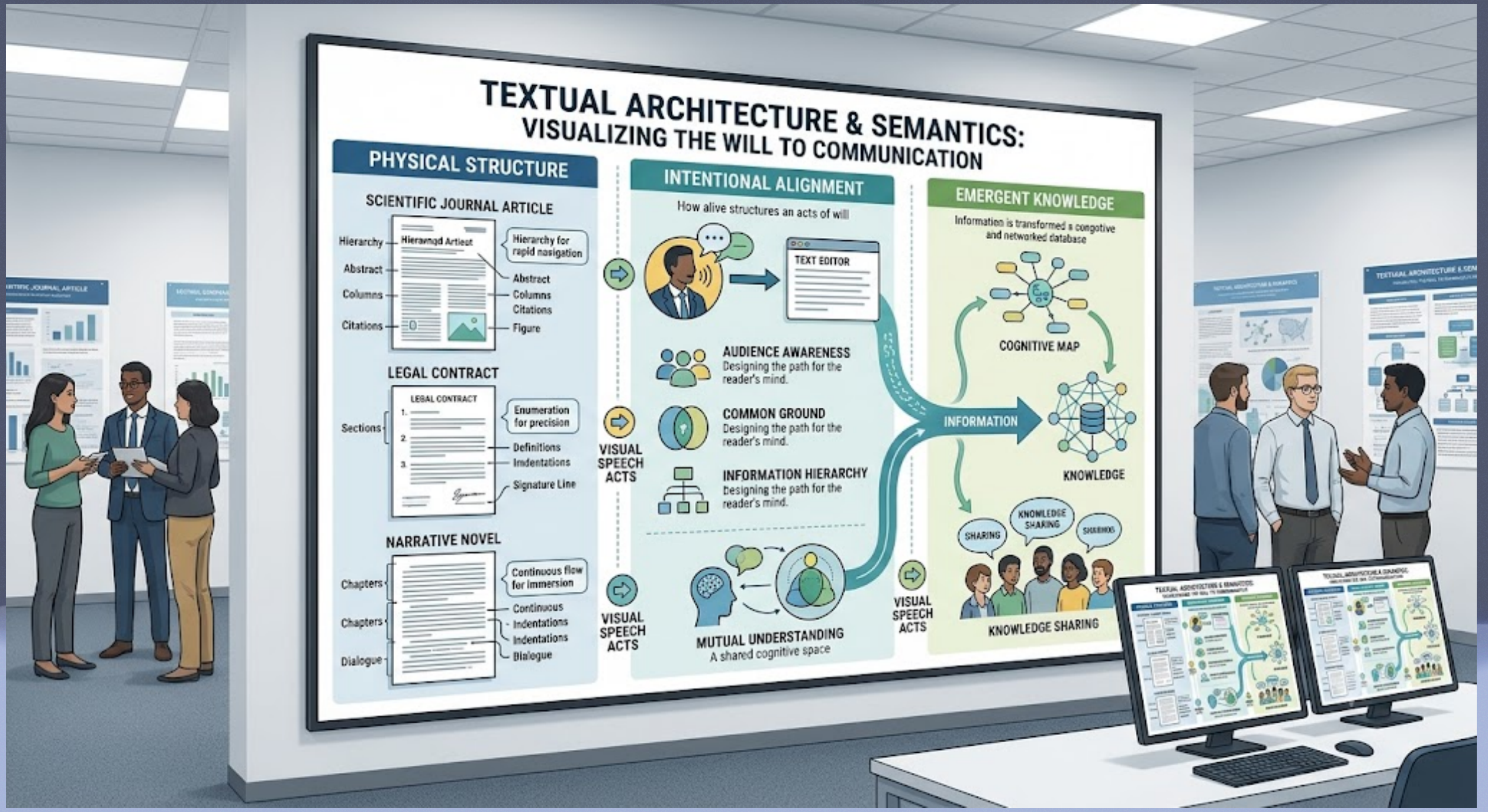
"STRUCTURE IS MEANING"

SHARED AGREEMENT



CONTEXTUAL LOGIC

CoreQuarry


TEXTUAL ARCHITECTURE & SEMANTICS:
VISUALIZING THE WILL TO COMMUNICATION
PHYSICAL STRUCTURE
SCIENTIFIC JOURNAL ARTICLE
Hierarchy
Abstract
Columns
Citations
Hierarchy for rapid navigation
Abstract
Columns
Citations
Figure
LEGAL CONTRACT
Sections
Enumeration for precision
Definitions
Indentations
Signature Line
NARRATIVE NOVEL
Chapters
Chapters
Dialogue
Continuous flow for immersion
Continuous
Indentations
Indentations
Dialogue
INTENTIONAL ALIGNMENT
How alive structures an acts of will
TEXT EDITOR
AUDIENCE AWARENESS
Designing the path for the reader's mind.
COMMON GROUND
Designing the path for the reader's mind.
INFORMATION HIERARCHY
Designing the path for the reader's mind.
VISUAL SPEECH ACTS
VISUAL SPEECH ACTS
MUTUAL UNDERSTANDING
A shared cognitive space
EMERGENT KNOWLEDGE
Information is transformed a congotive and networked database
COGNITIVE MAP
INFORMATION
KNOWLEDGE
SHARING
KNOWLEDGE SHARING
KNOWLEDGE SHARING

Deux passages très déterminés, partiels, particuliers, deux exemples. Mais de l'essence l'exemple se joue peut-être.

Premier passage : la religion des fleurs. Dans la *Phénoménologie de l'esprit*, le développement de la religion naturelle a comme toujours la forme d'un syllogisme : le moment médian, « la plante et l'animal », compose une religion des fleurs. Celle-ci n'est pas même un moment, une station. Elle s'épuise presque dans un passage *(Übergehen)*, un mouvement évanouissant, l'effluve flottant au-dessus d'une procession, la marche de l'innocence à la culpabilité. La religion des fleurs serait innocence, la religion des animaux coupable. La religion des fleurs (l'exemple facile) en viendrait d'Afrique, mais surtout de l'Inde) ne reste pas, ou à peine, elle procède à sa propre mise en culpabilité, à sa propre animalisation, au devenir coupable et donc sérieux de l'innocence. Et cela dans la mesure où le même, le soi-même *(Selbst)* n'y a pas encore lieu, ne se donne, encore, que (dans) sa représentation *(Vorstellung)*. « L'innocence de la religion des fleurs, qui est seulement représentation de soi-même sans le soi-même, passe dans le sérieux de la vie agonistique, dans la culpabilité de la religion des animaux; la quiétude et l'impuissance de l'individualité contemplative passe dans l'être-pour-soi destructeur. »

« Die Unschuld der Blumenreligion, die nur selbstlose Verteilung des Selbstlosen, geht in den Ernst des kämpfenden Lebens, in die Schuld der Tierreligion, die Ruhe und Ohnmacht der anschauenden Individualität in das zerstörende Fürsichsein über. »

Deuxième passage : la colonne phallique de l'Inde. L'*Esthétique* en décrit la forme au chapitre de l'*Architecture symbolique*. Elle se serait propagée vers la Phrygie, la Syrie, la Grèce où, comme des titres élogieux, selon Hérodote cité par Hegel, les femmes tressaient le fil d'un phallus qui se dressait alors en l'air, « presque aussi grand que le reste du corps ». À l'origine, donc, les colonnes phalliques de l'Inde, énormes formations, piliers, tours, plus larges

L'autre — laisse encore le reste. Risquant de revenir au même. Tombe — deux fois les colonnes, les tombes — reste.

Peut-être le cas *(Fall)* du sein.

Si *Fall* marque le cas, la chute, la décadence, la faillite ou la fesse, *Falle* égale piège, trappe, collet, la machine à vous prendre par le cou.

Le sein tombe.

Le reste est indicible, ou presque : non par approximation empirique mais à la rigueur indécidable.

« Geschwätze, s.f. I. Propos par lequel on met délicatement de côté cette propre est accepté dans le langage courtisan pour désigner une autre chose qui a quelque arabesque avec l'objet. Qu'il exprimant d'abord : par exemple, une langue, parce que la langue est le par notre organe de la parole artic[illegible] une glace [...] une feuille de papier [...]. C'est aussi par catachrèse qu'on dit : fleuri d'argent ; aller à cheval sur un bâton, [...] 2. Terme de musique. Dissonance dure et insolite. F. Kunstgeschw. eines, de vanité, contre, et γράφειν, vitage.

Catéchèse, s.m. Entraide literale, par honneur, au nid ou, d'une église, pour recevoir le catéchisme ou la répétition [...] 2. Rel. catéchèse, catéchèse, catéchèse, catéchèse, catéchèse, catéchèse. Cote est selon Dr. Cange le bastion carré, machine de guerre appelée chez d'après l'animal, et selon Chez même, voir, regarder : du reste, finalement, ces deux étymologies se confondent, car que cette, chat, et nature, regarder, ont la même racine. Reste facile, qui, en les variantes du nombre où le p se montre, ne peut être que le mot germanique *faih* (esp. *kakeos*). Catéchèse est le même mot que échafaud (esp. id. mot).

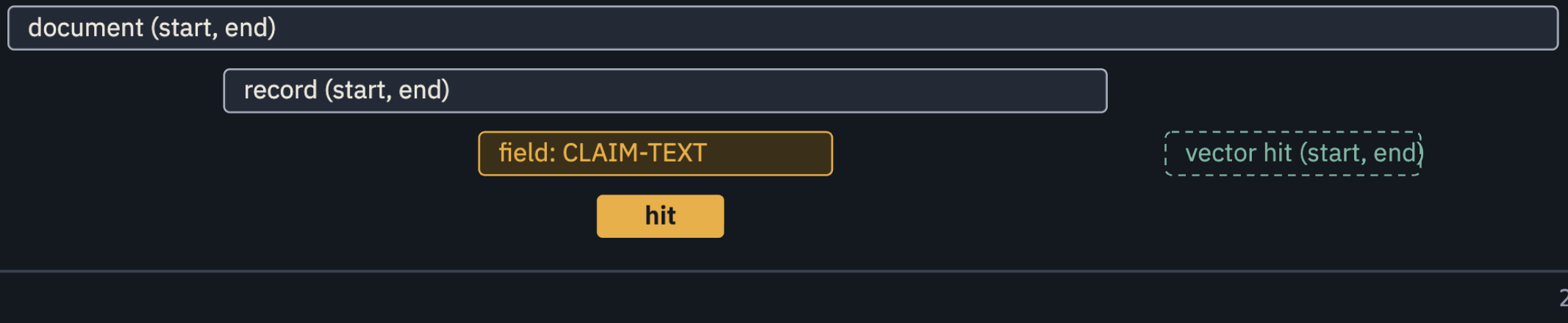
Catapétisme, s.m. Terme de littérature ancienne. Emploi de mots rechercher chés. E. Kunstkatapult, de κατά, indiquant recherche, et πέλαμα, mot, langue (esp. *gheas*). a Litré.

Les ALC sonnent, claquent, éclatent, se réfléchissent et se retournent dans tous les sens, comptent et se dénombrent, ouvrant — ici — dans la pierre de chaque colonne des sortes de jolies incroyable, crénelées, jalousies, meurtrières pour voir à ce pas se laisser emprisonner dans la colonne, laissées dans la peau pilade d'un corps

מאימתי קורין את שמע בערבין ◦ משעה שהכהנים נכנסים לאכול בתרומתן עד סוף האשמורה הראשונה דברי רבי אליעזר חכמים אומרים ◦ עד חצות רבן גמליאל אומר ◦ עד שיעלה עמוד השחר מעשה ובאו בניו מבית המשתה אמרו לו לא קרינו את שמע אמר להם אם לא עלה עמוד השחר חייבין אתם לקרות ולא זו בלבד אמרו אלא כל מה שאמרו חכמים עד חצות מצותן עד שיעלה עמוד השחר ◦ יהקטר חלבים ואברים מצותן עד שיעלה עמוד השחר ◦ וכל הנאכלים ליום אחד מצותן עד שיעלה עמוד השחר אם כן למה אמרו עד חצות ◦ לדרדק אדם מן העבירה : נמי יתנא הכא קאי דקתני מאימתי ותו מאי שנא דתני בערבית ברישא קאי ◦ דיכתיב ◦ בשכבך וכקומך והכי קתני זמן קריאת שמע דשכיבה אימת משעה שהכהנים נכנסין לאכול בתרומתן ואי בעיא איכא למיפרך של עתים דכתיב ◦ ידקתני ◦ בשעה שהיו מביאין את הבכורים ◦ דכתיב ◦ ויהי בקר יום אחד אי הכי סיפא ◦ לאחריה ◦ בשעה שהיו מביאין את הבכורים ◦ דכתיב ◦ ויהי ערב ויהי בקר יום אחר אי הכי סיפא ◦ לאחריה ליתני דערבית ברישא הנא פתח בערבית והדר תני בשחרית עד דקאי בשחרית פריש מילי דשחרית והדר פריש מילי דערבית ; אטו כל משעה שהכהנים נכנסים לאכול בתרומתן מבני בתים אינה אלא אכלי תרומה משעת צאת הכוכבים ליתני משעת צאת הכוכבים אימא קא אכלי בתרומה משעת צאת הכוכבים ◦ כדתניא ◦ כל הנאכל לכהנים משעת צאת הכוכבים ◦

THE IDEA

One address space. Everything else is a span.



Structure is arithmetic on intervals. A vector hit is just another interval.

THE IDEA

An old idea, continued.

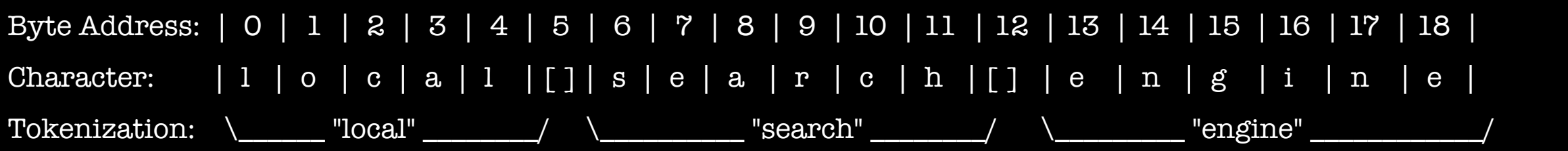
PAT arrays	Gonnet: index every text position, search by prefix on the raw string.
Region algebras	Queries as operations on intervals: inside, containing, followed by.
HyTime	Addressing by location, not by copying content.
CoreQuarry	One global address space, fields as spans, vector hits as spans, on commodity hardware.

"Dynamic units of retrieval: spans, evidence regions, and the algebra an agent actually needs."

Patricia Arrays

- Gonnet, G. H., Baeza-Yates, R., & Snider, T. (1992). New indices for text: PAT trees and PAT arrays. In Information Retrieval: Data Structures & Algorithms (pp. 66-82). Prentice-Hall, Inc.
- Baeza-Yates, R., Barbosa, E. F., & Ziviani, N. (1996). Hierarchies of indices for text searching. Information Systems, 21(6), 497-514. https://doi.org/10.1016/0306-4379(96)00025-7

Patricia Arrays: Basic Idea



A "Semi-Infinite String" is simply a pointer to a starting byte address, viewing the data from that byte all the way to the end of the buffer.

The array of addresses is then sorted alphabetically.

Inverted indexes and PAT arrays

The same occurrences, organized by term or by the text that follows.

SHARED SOURCE

D1	r	e	d		f	o	x		r	e	d		h	e	n
	0				4				8				12		

D2	r	e	d		f	o	x
	0				4		

Zero-based byte offsets

Suffixes end at document boundaries.

Positional inverted index

Term → occurrence postings

TERM DICTIONARY	POSTINGS: docID → byte offsets
fox →	D1: [4] D2: [4]
hen →	D1: [12]
red →	D1: [0,8] D2: [0]

Phrase search combines the positional postings.

```
fox_start = red_start + 4        in this example
```

"red fox" → D1@0, D2@0

Postings can also carry frequency, weights and other payloads.

PAT array

Text-start pointers in lexicographic suffix order

RANK	POINTER	TEXT AT POINTER (preview)
0	D2@4	fox
1	D1@4	fox red hen
2	D1@12	hen
3	D2@0	**red fox**
4	D1@0	**red fox** red hen
5	D1@8	red hen

"red fox" contiguous match range

Binary-search the prefix interval.

"red fox" → D2@0, D1@0

Array stores pointers; the previews are read from the source.

INVERTED INDEX

Term / set queries → postings operations

Phrases and proximity → positional joins

PAT ARRAY

Literal prefixes / phrases → a sorted range

Substring coverage depends on the indexed start positions.

Group by the term, or order by the text that follows.

This PAT example indexes all six token starts. A full suffix array can index every character position. Source suffixes are not copied into the array.

Manning, Raghavan & Schütze (2008), Introduction to Information Retrieval.

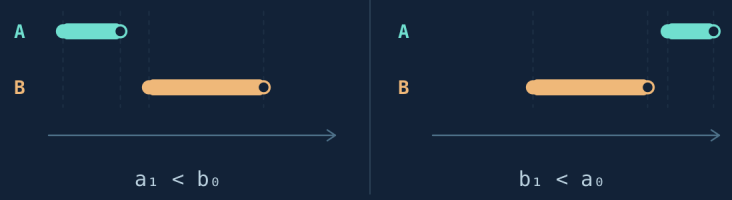
Gonnet, Baeza-Yates & Snider (1992), New Indices for Text: PAT Trees and PAT Arrays.

Allen’s interval algebra

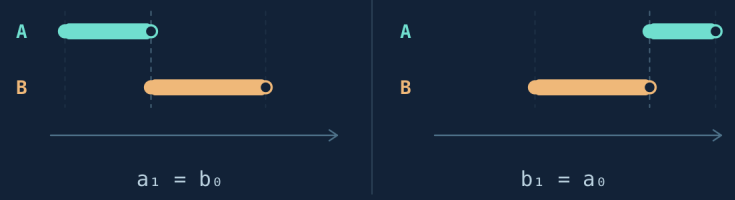
13 basic relations for proper intervals — on time or source-position axes.

$A = [a_0, a_1)$ $B = [b_0, b_1)$ $a_0 < a_1$, $b_0 < b_1$

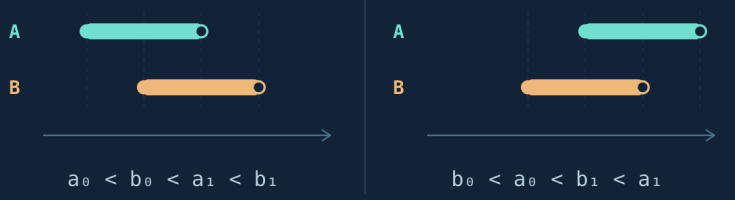
Before ↔ After



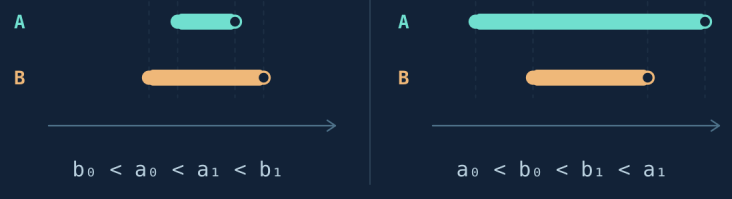
Meets ↔ Met by



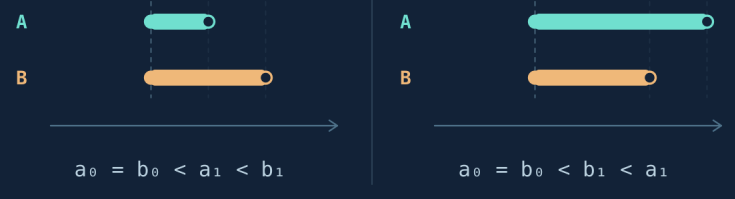
Overlaps ↔ Overlapped by



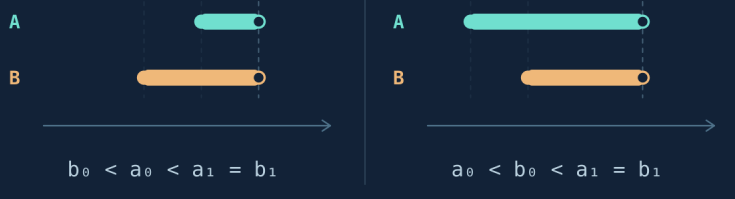
During ↔ Contains



Starts ↔ Started by



Finishes ↔ Finished by



Equals

$a_0 = b_0$ and $a_1 = b_1$

6 converse pairs + equality = 13

TEXTUAL SPANS

`[s,e] → [s,e+1)`

Inclusive byte positions → half-open boundaries

MEETS IN TEXT

`A.end + 1 = B.start`

Adjacent spans; no intervening byte

RELATIONS ALSO COMPOSE

A before B, B before C

⇒ A before C

James F. Allen (1983). “Maintaining Knowledge about Temporal Intervals.”

Communications of the ACM 26(11), 832–843. DOI: 10.1145/182.358434

Every pair of definite proper intervals has exactly one basic relation. Diagrams compare A with B; a_0/b_0 are starts and a_1/b_1 are end boundaries.

Patricia Trees and Pat Arrays Fell Out of Favor in the 1990s because of hardware:

- Pointer chasing is catastrophically cache-unfriendly— every node traversal is a potential cache miss.
- On 1990s hardware, a cache miss meant going to main memory at 60–80ns, and with tiny caches (L1 was 8–16KB) misses were constant.
- The inverted index, despite its bulk, had more predictable, sequential access patterns that could be optimized with careful disk layout.
- Patricia trees also consumed significant RAM for pointers — precious in an era of $50,000/GB RAM.
- Worst case pointer density: a tree over a million terms might have millions of pointer pairs, each potentially a cache miss during traversal

Its 2026. Not 1999. What Contemporary Hardware Means for Information Retrieval.

The most dramatic difference isn't raw speed — it's random I/O latency and IOPS, which is exactly what database queries and information retrieval hammers hardest.

- A full-table scan that took 10 minutes on a 90s SCSI array takes seconds on NVMe.
- A database query requiring 10,000 random disk reads: 90s machine takes 80–120 seconds just in I/O wait; modern NVMe does it in ~10–50 milliseconds.
- The entire working dataset that required careful buffer pool management in the 90s (because RAM was scarce) now fits entirely in RAM on a modern system — eliminating disk I/O almost entirely for hot data.
- Memory bandwidth improvements mean in-memory analytics (columnar stores, etc.) can process data ~750× faster even if latency is similar.

Cache hierarchies have transformed things

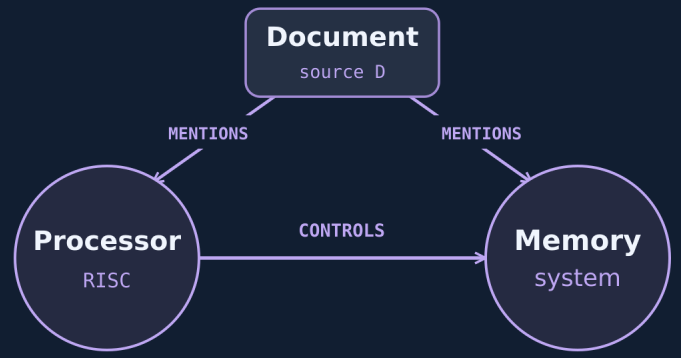
Cache	High End 1996 Size	COTS 2026 Size	Improvement
L1	8–16 KB	32–64 KB	~4×
L2	256 KB (rare)	512 KB–1 MB per core	~4–8×
L3	Nonexistent/tiny	64–256 MB (Threadripper)	∞ → huge
RAM	256MB–4GB	128GB–2TB	~500×

Textual graphs and entity graphs

Two complementary models of relationship

Entity / relationship graph

Neo4j property graphs · entity-centric GraphRAG



Entities connected by modeled relationships

Queries match graph patterns and follow paths.

Neo4j Graph database: stores and queries the graph

GraphRAG Builds / uses a graph for retrieval and synthesis

Illustrative entity model

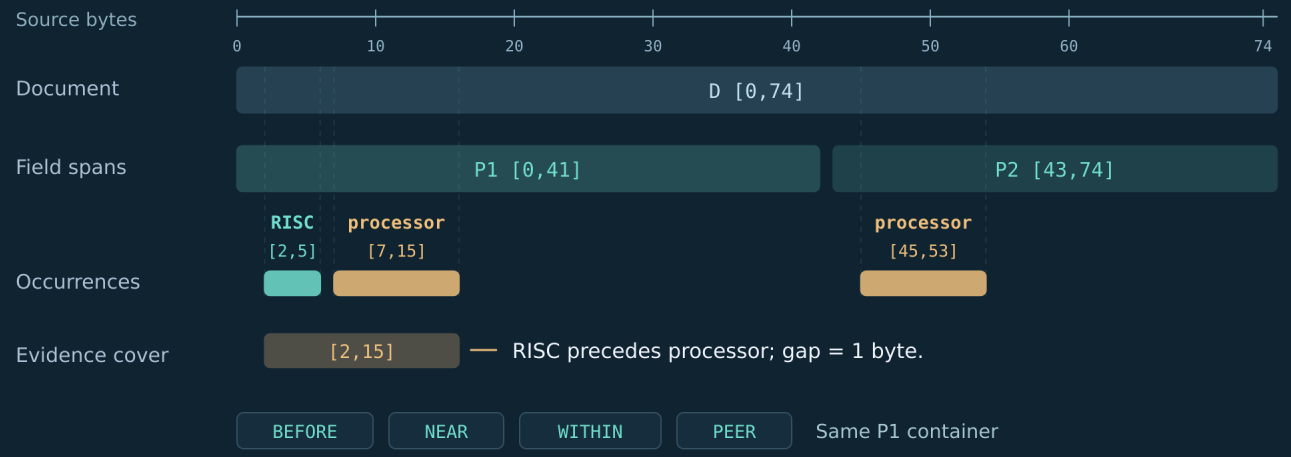
CoreQuarry textual graph

Occurrences and spans; relations derived from source coordinates

SOURCE D

P1 A RISC processor controls a memory system.

P2 A processor uses a write buffer.



HyTime FCS

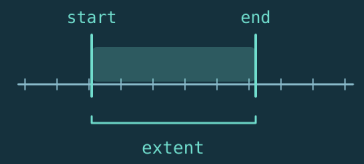
Finite Coordinate Space

ISO/IEC 10744

Position + extent → addressable spans

Text supplies a finite, one-dimensional source-position axis.

`[start, end] → extent = end − start + 1`



CoreQuarry makes textual structure queryable and evidence addressable.

Example: ASCII source, zero-based byte offsets, inclusive span ends; newline at byte 42.

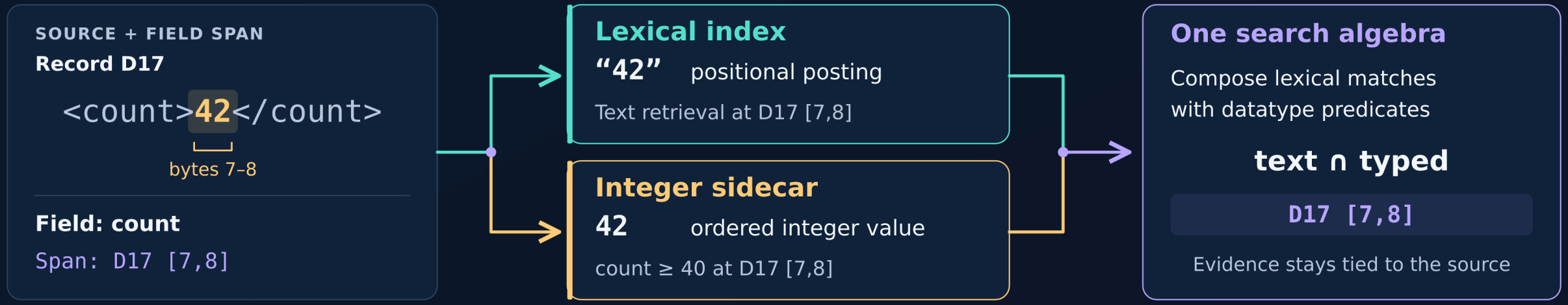
Comparison of modeling focus: entity graphs can also retain source links and spans.

References: Neo4j · Microsoft GraphRAG · HyTime / Newcomb

COREQUARRY

Datatype sidecars

One source span, indexed as text and as a typed value



Datatype families

Each value domain has its own search semantics

Integers

Signed 128-bit domain
Adaptive 32/128-bit storage

Decimals & floating point

128-bit fixed point, 16 fraction digits
IEEE 754 binary64 (float64)

Currency & Boolean

Fixed-point monetary values
Boolean fields

Dates & time

Dates and date ranges
Time, TTL and expiry

Numeric ranges

Exact decimal endpoints
Numeric interval search

Geospatial

Bounding boxes
N-ary geographic bounds

Phonetics & hashes

Phonetic and Metaphone
Hash, casehash and lexical hash

Vectors & computed values

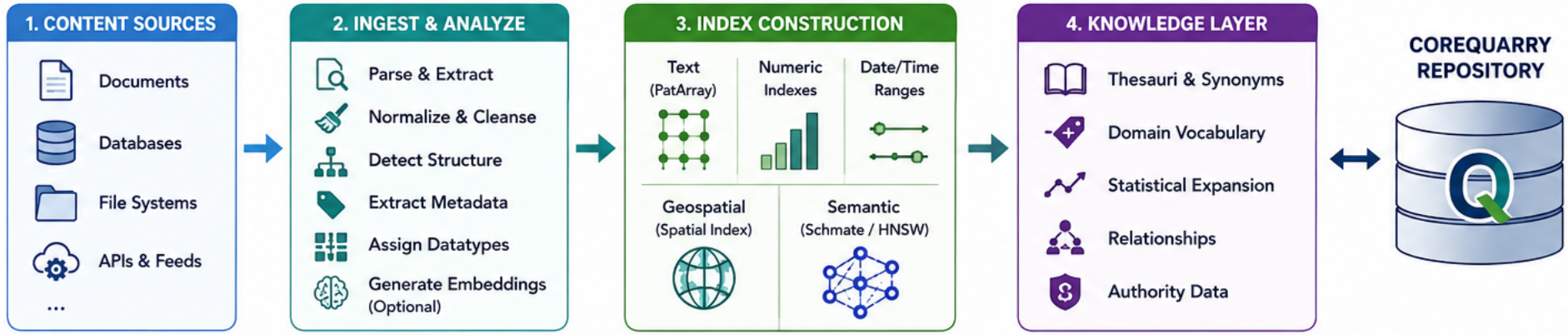
HNSW models and raw vectors*
Computed numerical values

Separate index structures. Common source coordinates.

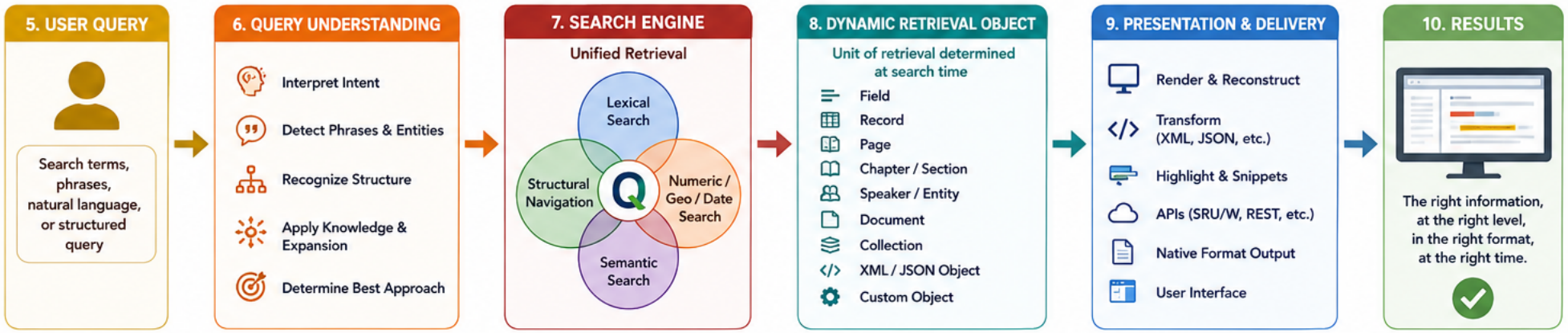
Illustrative byte positions are local and inclusive. Selected datatype families shown. *Vector support depends on build.

CoreQuarry High-Level Process Flow

From Content Ingestion to Dynamic Retrieval and Delivery



Content is transformed into rich, multi-dimensional indexes and knowledge for unified, high-performance retrieval.



CoreQuarry delivers the most appropriate information object — not just documents — based on query intent and document structure.

KEY BENEFITS

- Precision Retrieval
- Structural Awareness
- Any-to-Any Formats
- High Performance
- Extensible Framework
- Standards & Interoperability
- Domain Adaptable

DEMO · SHAKESPEARE

Stopwords are words. Search the phrase.

```
$ lsearch -d SHAKESPEARE -show “to be or not to be"

Query: "to be or not to be"

1 record of 37 matched your query, 1 record displayed.

       Score  File
   1.    100  /Volumes/Celeritas/CoreQuarry/ib/data/shakespeare.xml
`The Tragedy of Hamlet, Prince of Denmark’
Evidence: [PLAY\ACT\SCENE\SPEECH\LINE] To be, or not to be: that is the question:
```

~10 ms every word of it a stopword somewhere else

The index is 22,945 entries and about 0.67 MB.

DEMO · WIKIPEDIA (a 2 million record ES Bulk Dump)

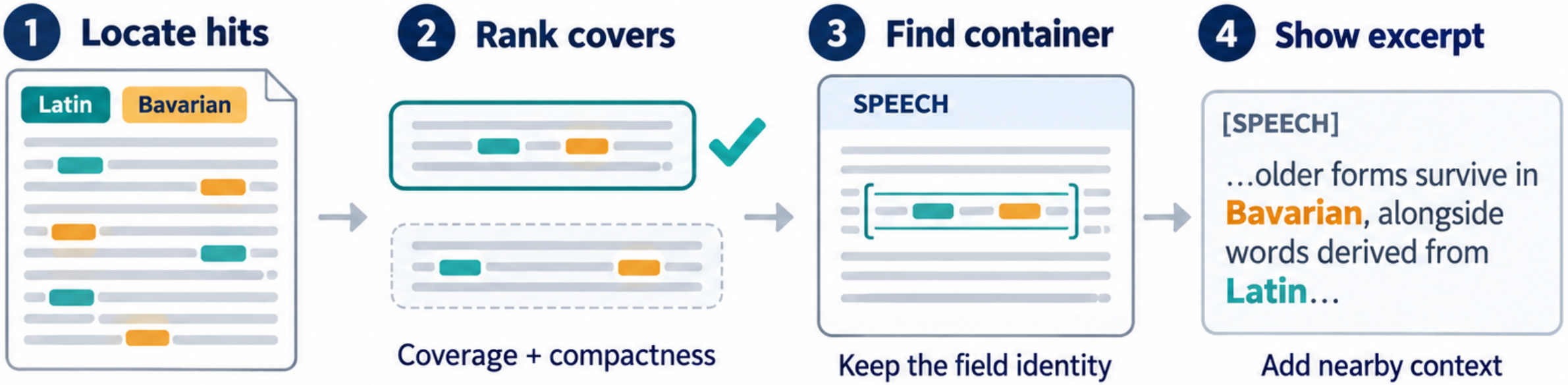
Phrase search

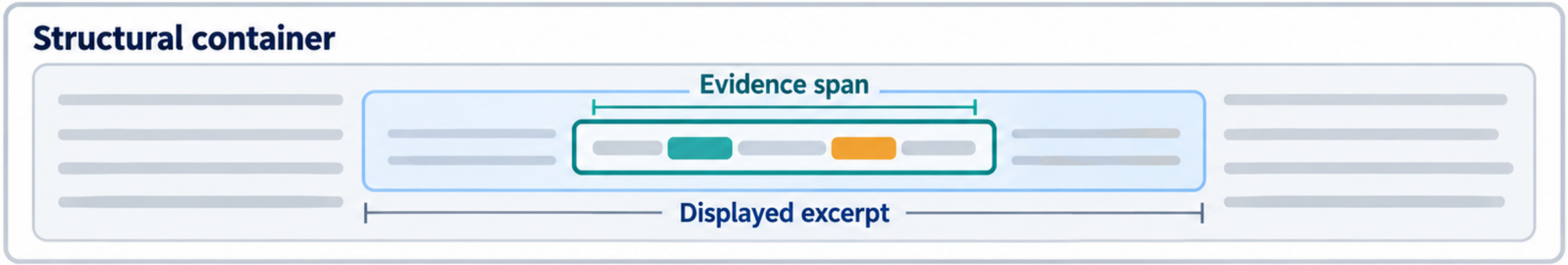
$/lsearch -d /Volumes/D50/indexes/WIKI_ALL -show is an error

```
Isearch [Info]: Phrase "is an error" uses all common words (131340 > 10000 : 500000)
[0]. Using alternative search heuristic: "is" "an" #< "error" #<
CPU Process time: 469 ms. (3) Search: 466 ms.
Query: "is an error"

290 records of 2065536 matched your query, 290 records displayed.

       Score  File
   1.    100  /Volumes/D50/Wikidata/afwiki-20251229-cirrussearch-general.json
`Citation/ CS1'
Evidence: [SOURCE_TEXT] age.
 See also coins_cleanup().

 Output of this function is an error message that identifies the character or the
 Unicode group
```

IB: From query hits to evidence

Coverage + compactness + document structure





Byte advice is a soft budget; selected evidence stays intact.

IB: How evidence covers are scored

Coverage rewards completeness. Compactness rewards concentration.

Coverage (Energy)

$$C = \frac{n}{N}$$

n = covered dimensions

Dispersion

$$d = \frac{W}{n}$$

N = represented dimensions

Compactness

$$K = \frac{1}{1 + d/\delta}$$

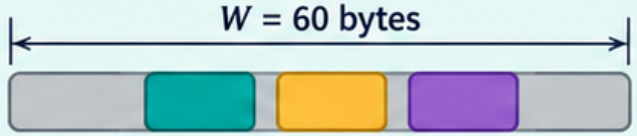
W = span in bytes

Cover rank: $R = C^{\alpha} \times K^{\beta}$

- α EvidenceWeight = 1.5
- β DispersionWeight = 1.0
- δ DispersionDecay = 32 bytes per dimension

Example: $N = 3$ evidence dimensions (schematic spans)

Compact + complete ✓



$n = 3$ $W = 60$ bytes
$C = 1.0000$ $d = 20$

$R = 0.6154$

Compact + partial

$n = 2$ $W = 20$ bytes
$C = 0.6667$ $d = 10$

$R = 0.4147$

Wide + complete

$n = 3$ $W = 300$ bytes
$C = 1.0000$ $d = 100$

$R = 0.2424$

Display selection: subtract the attention cost

$$U = R - \lambda \times [\max(0, (W - B)/B)]^{p}$$

- B MaxBytesAdvice = 130 bytes (Context default)
- λ DisplayOverageWeight = 0.25
- p DisplayOverageExponent = 2.0
- DisplayHardLimitFactor = 4.0 · Guardrail beyond 4B

For the 300-byte cover:

$U = 0.2424 - 0.4275 = -0.1851$

No overage cost when $W \le B$

Current source defaults • $B = 0$ disables attention cost • Structure is applied after evidence selection

DEMO · SHAKESPEARE

Same evidence, two extents.

EXTENT: THE LINE

[query + top hit]

EXTENT: THE SPEECH

[same query, wider extent, top hit]

```
`The Tragedy of Hamlet, Prince of Denmark'
  [1] "speech: <SPEAKER>HAMLET</SPEAKER>
<LINE>To be, or not to be: that is the question:</LINE>
<LINE>Whether 'tis nobler in the mind to suffer</LINE>
<LINE>The slings and arrows of outrageous fortune,</LINE>
<LINE>Or to take arms against a sea of troubles,</LINE>
<LINE>And by opposing end them? To die: to sleep;</LINE>
<LINE>No more; and by a sleep to say we end</LINE>
```

Which extent counts is a choice at query time, not an index you had to build first.

DEMO · SHAKESPEARE

Structure on demand, not in advance.

$ [query constrained to a speaker or a scene]

lsearch -d SHAKESPEARE -XML -P speech/speaker to be or not

→ [the spans returned, with their start–end addresses]

Speech was never indexed as a record. Line, speech, scene and act are just spans (intervals), so the engine just does a comparison of spans to reconstruct in this case the tree (graph) connect to a hit.

'ACT' 'EPILOGUE' 'FM' 'GRPDESCR' 'INDUCT' 'LINE' 'P' 'PERSONA' 'PERSONAE' 'PGROUP' 'PLAY' 'PLAYSUBT' 'PLAY\ACT' 'PLAY\ACT\EPILOGUE'
'PLAY\ACT\EPILOGUE\SPEECH' 'PLAY\ACT\EPILOGUE\SPEECH\LINE' 'PLAY\ACT\EPILOGUE\SPEECH\SPEAKER' 'PLAY\ACT\EPILOGUE\STAGEDIR'
'PLAY\ACT\EPILOGUE\SUBTITLE' 'PLAY\ACT\EPILOGUE\TITLE' 'PLAY\ACT\PROLOGUE' 'PLAY\ACT\PROLOGUE\SPEECH' 'PLAY\ACT\PROLOGUE\SPEECH\LINE'
'PLAY\ACT\PROLOGUE\SPEECH\SPEAKER' 'PLAY\ACT\PROLOGUE\SPEECH\STAGEDIR' 'PLAY\ACT\PROLOGUE\SPEECH\SUBHEAD' 'PLAY\ACT\PROLOGUE\STAGEDIR'
'PLAY\ACT\PROLOGUE\TITLE' 'PLAY\ACT\SCENE' 'PLAY\ACT\SCENE\SPEECH' 'PLAY\ACT\SCENE\SPEECH\LINE' 'PLAY\ACT\SCENE\SPEECH\LINE\STAGEDIR'
'PLAY\ACT\SCENE\SPEECH\SPEAKER' 'PLAY\ACT\SCENE\SPEECH\STAGEDIR' 'PLAY\ACT\SCENE\SPEECH\SUBHEAD' 'PLAY\ACT\SCENE\STAGEDIR'
'PLAY\ACT\SCENE\SUBHEAD' 'PLAY\ACT\SCENE\TITLE' 'PLAY\ACT\TITLE' 'PLAY\FM' 'PLAY\FM\P' 'PLAY\INDUCT' 'PLAY\INDUCT\SCENE' 'PLAY\INDUCT\SCENE\SPEECH'
'PLAY\INDUCT\SCENE\SPEECH\LINE' 'PLAY\INDUCT\SCENE\SPEECH\SPEAKER' 'PLAY\INDUCT\SCENE\SPEECH\STAGEDIR' 'PLAY\INDUCT\SCENE\STAGEDIR'
'PLAY\INDUCT\SCENE\TITLE' 'PLAY\INDUCT\SPEECH' 'PLAY\INDUCT\SPEECH\LINE' 'PLAY\INDUCT\SPEECH\SPEAKER' 'PLAY\INDUCT\STAGEDIR' 'PLAY\INDUCT\TITLE'
'PLAY\PERSONAE' 'PLAY\PERSONAE\PERSONA' 'PLAY\PERSONAE\PGROUP' 'PLAY\PERSONAE\PGROUP\GRPDESCR' 'PLAY\PERSONAE\PGROUP\PERSONA'
'PLAY\PERSONAE\TITLE' 'PLAY\PLAYSUBT' 'PLAY\PROLOGUE' 'PLAY\PROLOGUE\SPEECH' 'PLAY\PROLOGUE\SPEECH\LINE' 'PLAY\PROLOGUE\TITLE' 'PLAY\SCNDESCR'
'PLAY\TITLE' 'PLAY\' 'PROLOGUE' 'SCENE' 'SCNDESCR' 'SPEAKER' 'SPEECH' 'STAGEDIR' 'SUBHEAD' 'SUBTITLE' 'TITLE'

COREQUARRY • DYNAMIC SEARCH-TIME UNIT OF RETRIEVAL

The unit of retrieval is determined by the query, not by the index.

SOURCE CONTENT

HTML
XML
SGML
PDF
Email
MARC
JSON
...

DOCTYPE REGISTRY & DOCUMENT HANDLERS

Parse • Extract • Present • Convert • Navigate

- Built-in Handlers
- Plug-in Handlers
- Structure Discovery
- Presentation & Conversion

STRUCTURAL OBJECT MODEL

Nodes • Fields • Containers • Relationships

- Ancestors
- Descendants
- Peers
- Siblings
- Containers

DATATYPE FRAMEWORK (Information Model)

- Text
- Numeric
- Date / Time
- Range
- Geospatial
- Embeddings (Schmate)
- ... Other Extensible Types

SPECIALIZED INDEXES (Each datatype uses its optimal index)

PatArray (Full-Text) | Numeric Index | Range Index | Spatial Index | Schmate (HNSW)

QUERY ALGEBRA & LANGUAGE

Infix • RPN • SRU/W • Z39.50

- Boolean Operators (AND, OR, NOT, XOR...)
- Positional Operators (ADJ, NEAR, BEFORE, AFTER...)
- Structural Operators (PEER, WITHIN, INSIDE, SIBLING, ANCESTOR...)
- Semantic Operators (FUZZY, PHONETIC, EMBEDDING, THESAURUS...)
- Unary Operators (NOT, WITHIN:field, REDUCE, TRIM, BOOST, SORTBY...)

EXAMPLE: SHAKESPEARE SEARCH

User Input (unknown structure)

out out spot

Smart Query Interpretation

Detects phrase / proximity / structure

out PEER out PEER spot

Query Execution

Finds terms as PEERS in same node

Structural Navigation

Line (hit) → Speech → Speaker

Result (dynamic unit)

speech/speaker
LADY MACBETH

Hit text: *"Out, damned spot! out, I say!--One: two: why,"*

RETRIEVAL ENGINE

1. Candidate Generation
2. Evaluate Constraints
3. Structural Navigation
4. Dynamic Unit Selection
5. Reconstruction & Transformation
6. Scoring & Ranking

Hit → Node → Ancestor → Object → Presentation

DYNAMIC UNIT OF RETRIEVAL (Chosen at Search Time)

Field • Record • Line • Speech
Speaker • Scene • Chapter • Act
Page • Document • Collection
XML View • JSON Object • Custom

Selected by query, heuristics, or user request

STRUCTURAL NAVIGATION APIs

GetAncestorContent(nodeName)
GetPeerFc(hitGp)
GetNodeTree(hitFc)
GetSibling() GetDescendant()
GetContainer() GetPath() ...

Powerful navigation at search time without knowing the encoding

PRESENTATION & OUTPUT

Original Format | XML / JSON | Native Formats | Tabular View | Highlighting & Snippets | Export APIs

APPLICATIONS

Enterprise Search | Digital Libraries | Legal & eDiscovery | Research & Analytics | Semantic Search & RAG | Custom Solutions

KEY VALUE

- ✓ One platform, many datatypes, many indexes
- ✓ Search across structure, content & meaning
- ✓ Retrieve anything: a word, a line, a speech, a chapter, a document, or an entire collection
- ✓ Understand intent, not just keywords

CoreQuarry retrieves navigable document objects rather than documents — delivering the right information, at the right level, every time.

DEMO · SHAKESPEARE

Words miss paraphrase. Vectors don’t.

WORD SEARCH

[paraphrase query] → [miss]

Query: "whether" "it" II "is" II "better" II "to" II "live" II "or" II "die" II FOCUS:8

```
30 records of 37 matched your query, 30 records displayed.

      Score  File
   1.   100  /Volumes/Celeritas/CoreQuarry/ib/data/shakespeare.xml
`The Tragedy of Richard the Third'
Evidence: [PLAY\ACT\SCENE\SPEECH\LINE] O sir, it is better to be
brief than tedious. Show
   2.    96  /Volumes/Celeritas/CoreQuarry/ib/data/shakespeare.xml
`Measure for Measure'
Evidence: [PLAY\ACT\SCENE\SPEECH\LINE] Unfit to live or die: O
gravel heart!
```

VECTOR SEARCH

[same query] → [the line, as a span

```
[default32]> search whether it is better to live or die
  - [score=0.848796, sid=793, label=792, tokens=[0,28]] Shakespeare's Hamlet
pondered whether to be or not to be, that is the question in the most famous
soliloquy in English literature examining existence, suffering, and mortality
itself.
  - [score=0.828010, sid=769, label=768, tokens=[0,24]] Socrates declared that
the unexamined life is not worth living, emphasizing the importance of self-
reflection and philosophical inquiry in achieving a meaningful human existence.
  - [score=0.827503, sid=384, label=383, tokens=[0,7]] Better safe than sorry in
dangerous situations.
  - [score=0.815082, sid=795, label=794, tokens=[0,30]] Henry David Thoreau
advised to go confidently in the direction of your dreams and live the life you
have imagined, encouraging individuals to pursue authentic existence over
conformity and convention.
  - [score=0.811914, sid=773, label=772, tokens=[0,27]] Benjamin Franklin
observed that in this world nothing can be said to be certain except death and
taxes, wryly commenting on life's unavoidable realities with characteristic wit.
[default32]>
```

Both answers are spans in the same address space, so you can intersect them.

Figure 1. Schmate Search & Embedding Architecture

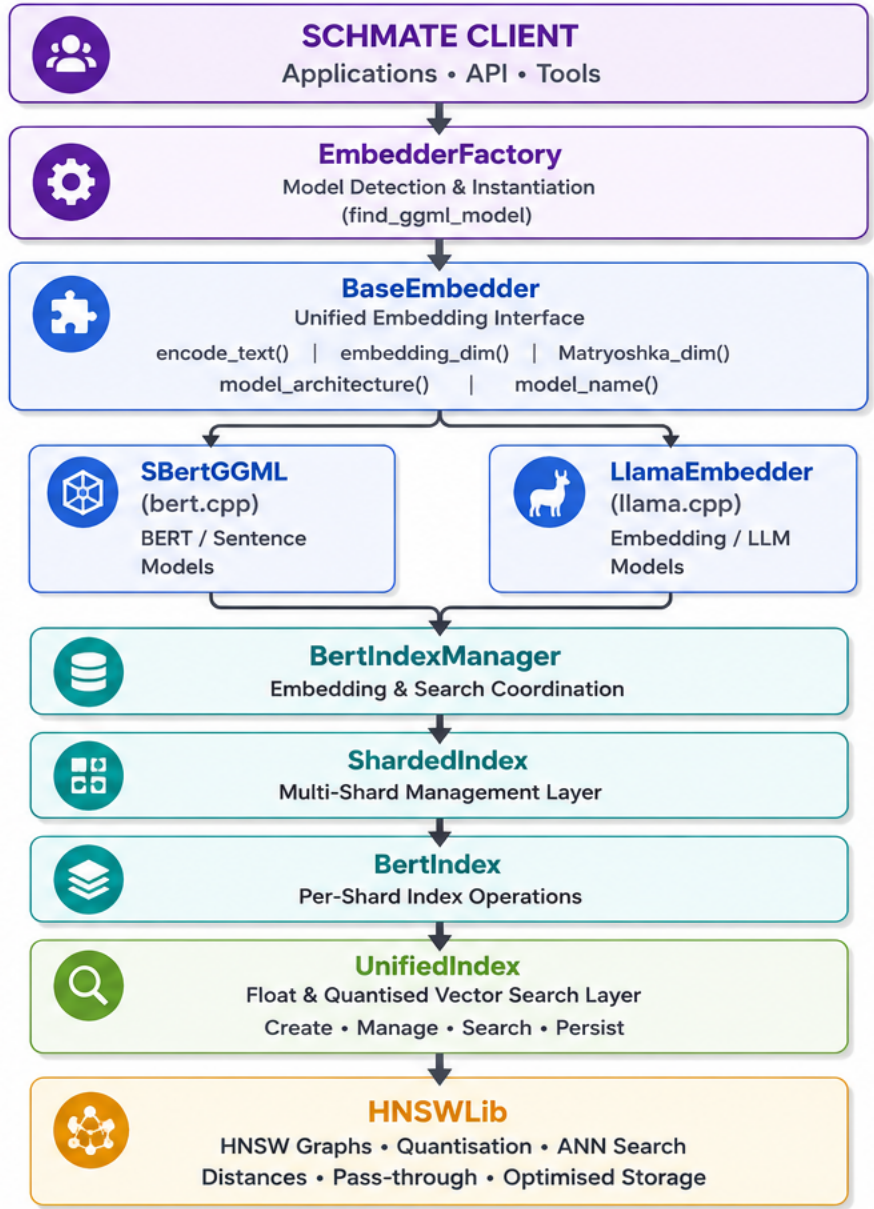
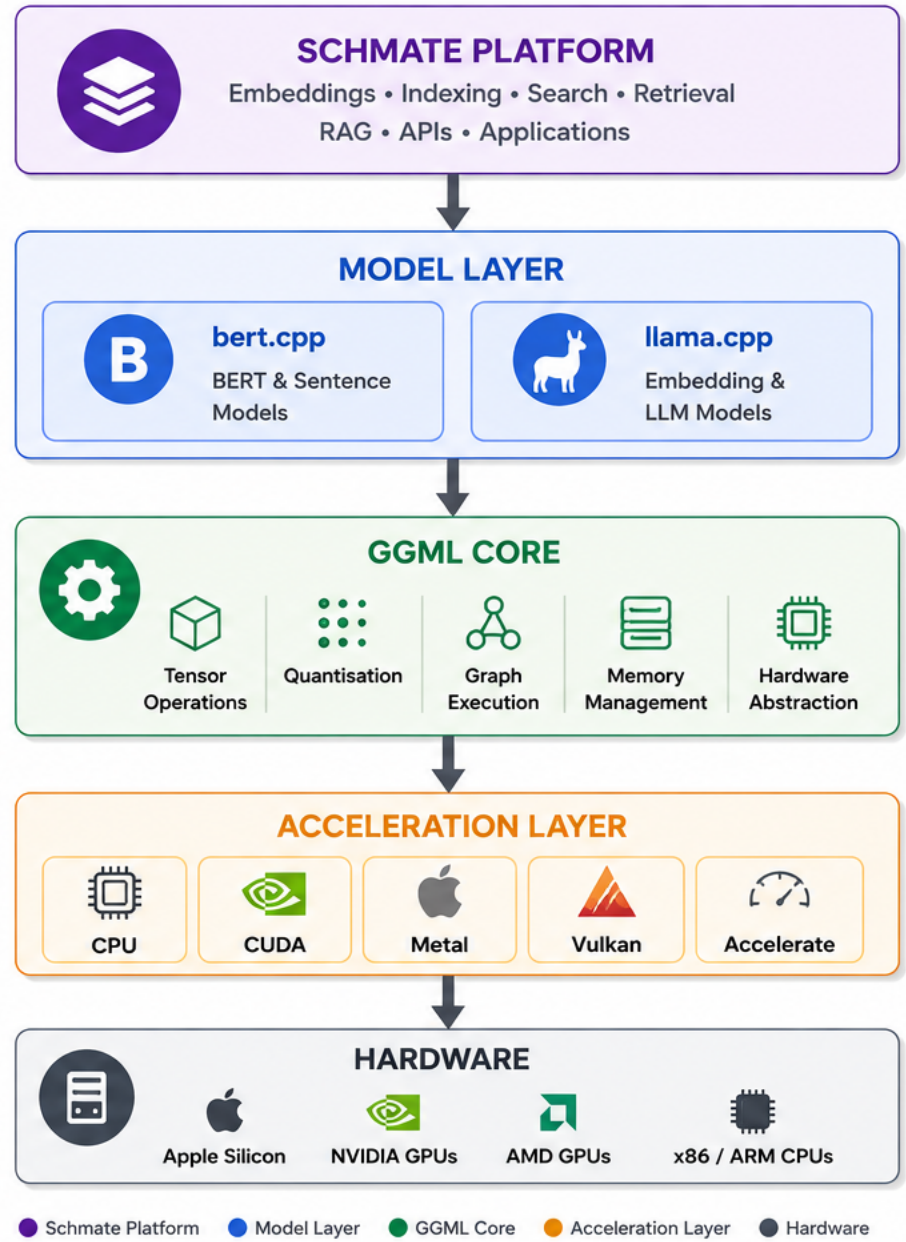


Figure 2. Schmate Runtime Stack



Schmate Storage & Quantization Pipeline

Flexible vector representations from direct pass-through to advanced quantization and binary encoding

Direct Pass-Through (No Quantization)
Quantization Path (Information Reduction)

1. INPUT / STORAGE REPRESENTATIONS (PASS-THROUGH SUPPORTED)

Schmate accepts embeddings in multiple native formats

Low Bit

BIN1	1-bit
INT2	2-bit
INT3	3-bit
INT4	4-bit
FP4	4-bit float
INT5	5-bit
INT6	6-bit

Standard / High Precision

INT8	8-bit
INT16	16-bit
FP16	16-bit float
BF16	16-bit (bfloat16) 8-bit exp / 7-bit mantissa
FLOAT32	32-bit float
FLOAT64	64-bit float (Not Supported)

Any supported format can be stored directly (**PASS**) with no information loss.

2. QUANTIZATION STRATEGIES

Choose the right algorithm for your accuracy / compression trade-off

PASS (Direct)
Store in original format. No quantization.

STANDARD
Balanced quantization using uniform or adaptive scaling.

BETTER
Improved accuracy with enhanced scaling techniques.

CENTROID
Cluster-based quantization using learned centroids.

ROTATIONAL
Rotational transform prior to quantization for lower error.

RABITQ
Binary quantization using rotation-aware bit allocation.

RABITQ EXTENDED
Enhanced RaBitQ with refined rotations & thresholds.

3. TARGET SEARCH REPRESENTATIONS (AFTER QUANTIZATION)

All quantization algorithms output one of the optimized search formats

BIN1 (Binary)
1-bit per dimension
(RaBitQ / RaBitQ Extended only)
- Maximum compression
- Highest index density

INT1.58
~1.58 bits per dimension
(Optimized ternary / multi-level encoding)
- Ultra-low memory
- High throughput

INT4
4 bits per dimension
- Excellent balance of accuracy & size

INT8
8 bits per dimension
- High accuracy with low memory

INT16
16 bits per dimension
- Highest accuracy (Minimal loss)

Choose the target that meets your accuracy, memory, and performance requirements.

VECTOR INDEX & SEARCH ENGINE (HNSW)

Fast similarity search over optimized representations

One Framework. Multiple Choices. Maximum Flexibility. Schmate uniquely supports both **direct storage** of pre-quantized embedings and **advanced quantization** of higher-precision vectors, using multiple algorithms and target formats to deliver the best trade-off for any use case.

FLEXIBLE INPUT
Accepts a wide range of native precision formats

ALGORITHM CHOICE
Multiple quantization strategies for any need

OPTIMIZED TARGETS
Multiple output precisions for search efficiency

PERFORMANCE
Higher index density, better cache utilization

EFFICIENCY
Lower memory, lower cost, higher scalability

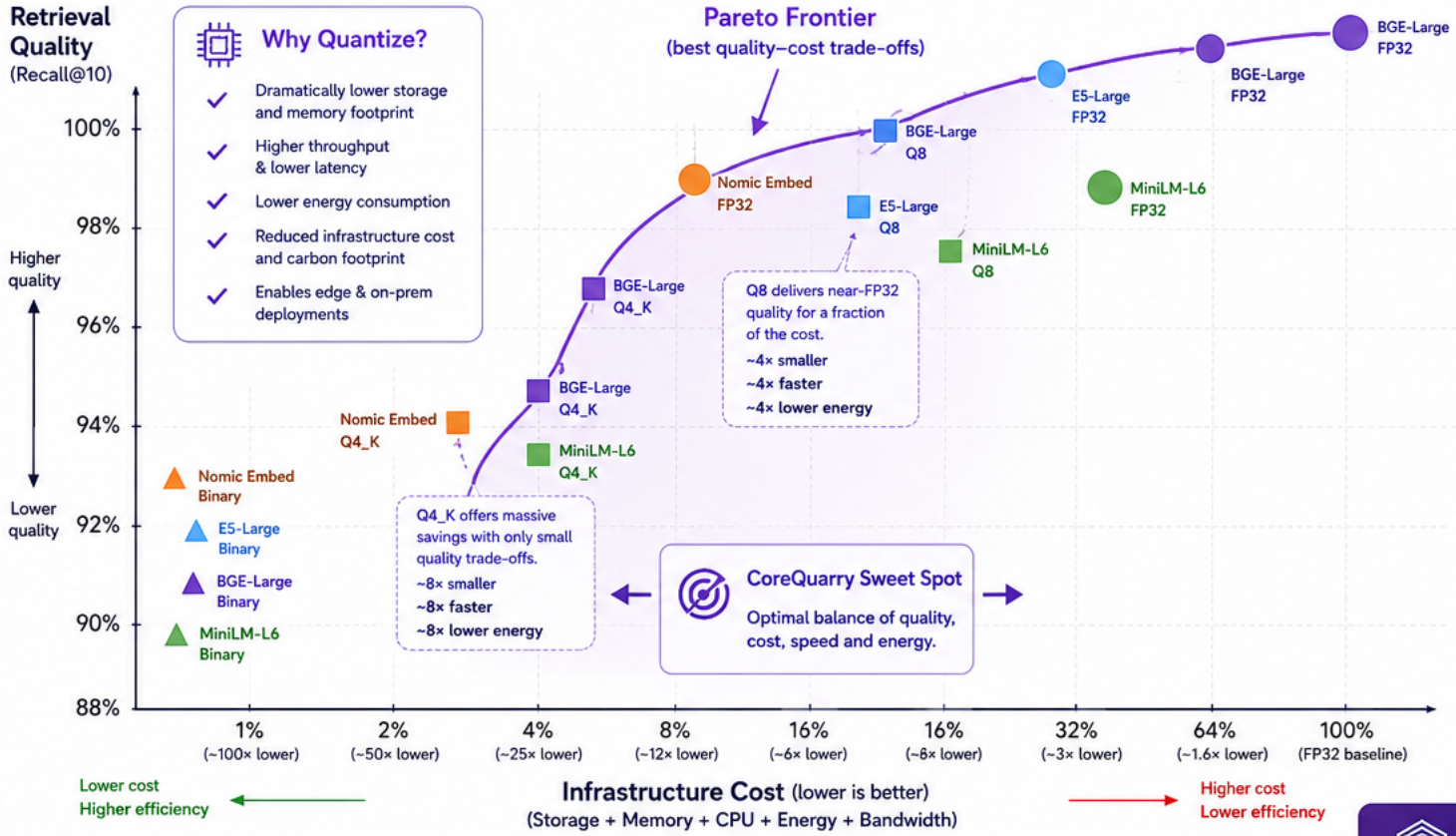
ACCURACY CONTROL
Choose the right balance for your application

ALGORITHM COVERAGE

- PASS works with all storage types
- All algorithms support INT16 / INT8 / INT4
- Standard / Better / Centroid / Rotational also support INT1.58
- RaBitQ and RaBitQ Extended output BIN1
- Always stored in optimal target format for search

The Retrieval Quality–Efficiency Frontier

Choose the optimal point for your workload. CoreQuarry supports every representation—natively.



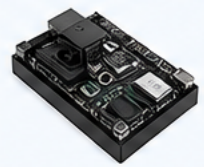
Energy & Sustainability

Efficient representations reduce compute, memory traffic and cooling requirements—cutting energy use and emissions.

Energy per 1M queries (typical)	vs FP32	CO_2e Impact (typical*)
FP32 (baseline)	100%	1.00×
FP16	~60%	~0.60×
Q8	~35%	~0.35×
Q4_K	~18%	~0.18×
Binary	~7%	~0.07×

*CO_2e impact varies by region and energy mix. Lower energy use directly reduces emissions.

On-Device Performance Example



NVIDIA Orin (MaxN @ 75W)
CoreQuarry + ggml / llama.cpp

8.5K tokens/s
@ ~75 W
≈ 113 tokens/s per Watt

- High throughput
- Low latency
- Low power
- Edge & offline ready
- Private & sovereign
- Lower TCO & carbon

Efficiency Gains by Representation (vs FP32 Baseline)

	FP32 (Baseline)	FP16	Q8	Q4_K	Binary / Multi-bit
Storage	100%	50%	25%	12.5%	3–4%
Memory	100%	50%	25%	12.5%	3–4%
Energy / query	100%	~60%	~35%	~18%	~7%
Throughput	1.0×	~1.5×	~2–3×	~3–5×	~8–15×

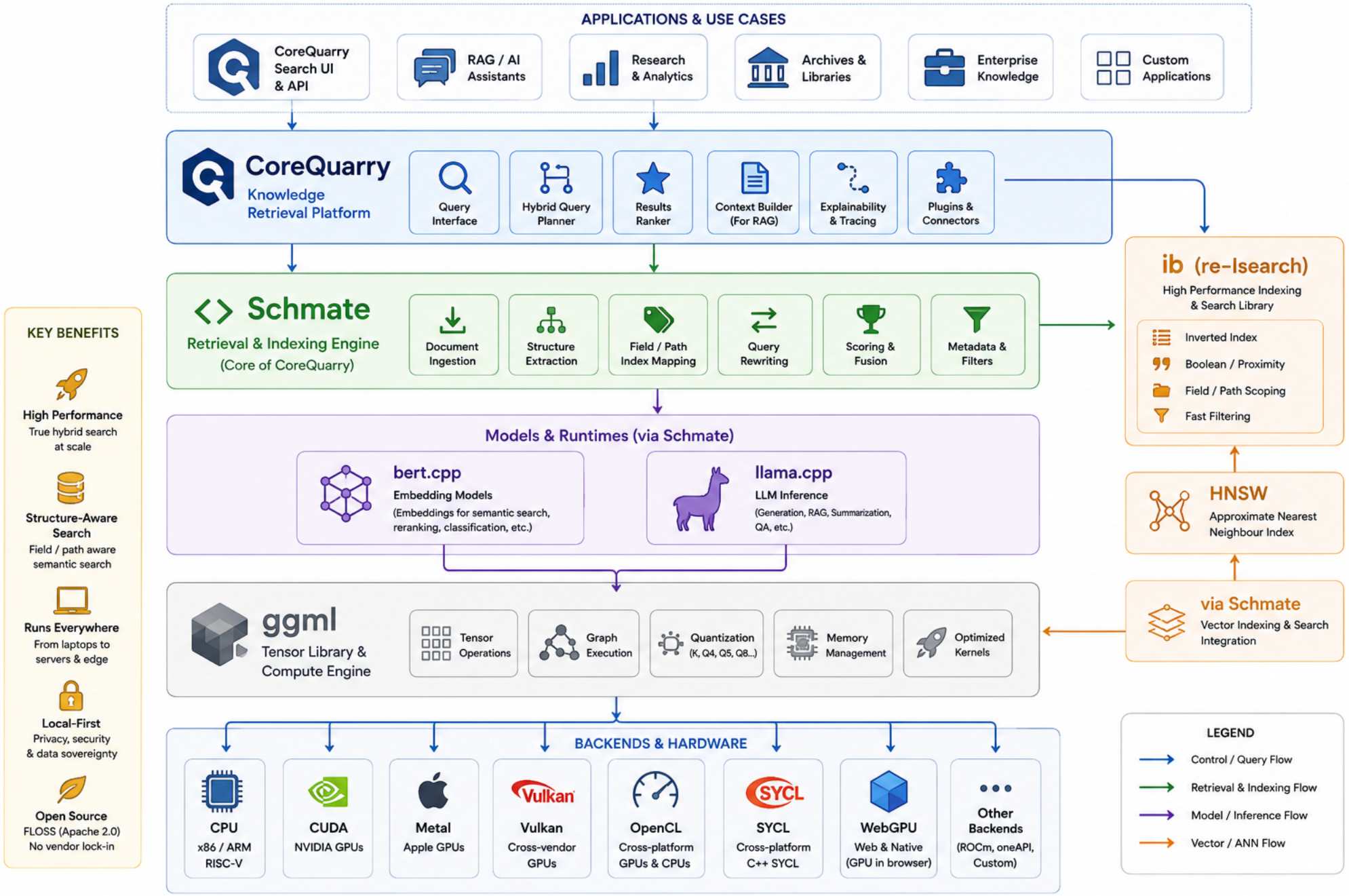
Key Takeaway

You can achieve near-FP32 quality while using up to 8–10× less storage, energy and compute.

Why CoreQuarry?

- Native support for float, quantized and binary embeddings
- Direct ingestion of pre-quantized models (no re-quantization)
- State-of-the-art quantization algorithms (including RaBitQ)
- Multi-representation indices in a single collection
- Highest efficiency search (across CPU, GPU, NPU)
- Optimal quality–cost–energy trade-offs
- Edge, on-prem and cloud ready
- Future-proof architecture for emerging embedding formats

Note: The values shown are indicative and will vary by dataset, hardware, implementation and evaluation setup. Use this chart as a guide to choose the right operating point for your application.


APPLICATIONS & USE CASES
CoreQuarry Search UI & API
RAG / AI Assistants
Research & Analytics
Archives & Libraries
Enterprise Knowledge
Custom Applications
CoreQuarry
Knowledge Retrieval Platform
Query Interface
Hybrid Query Planner
Results Ranker
Context Builder (For RAG)
Explainability & Tracing
Plugins & Connectors
Schmate
Retrieval & Indexing Engine
(Core of CoreQuarry)
Document Ingestion
Structure Extraction
Field / Path Index Mapping
Query Rewriting
Scoring & Fusion
Metadata & Filters
ib (re-lsearch)
High Performance Indexing & Search Library
Inverted Index
Boolean / Proximity
Field / Path Scoping
Fast Filtering
HNSW
Approximate Nearest Neighbour Index
via Schmate
Vector Indexing & Search Integration
Models & Runtimes (via Schmate)
bert.cpp
Embedding Models
(Embeddings for semantic search, reranking, classification, etc.)
llama.cpp
LLM Inference
(Generation, RAG, Summarization, QA, etc.)
ggml
Tensor Library & Compute Engine
Tensor Operations
Graph Execution
Quantization (K, Q4, Q5, Q8...)
Memory Management
Optimized Kernels
BACKENDS & HARDWARE
CPU
x86 / ARM RISC-V
CUDA
NVIDIA GPUs
Metal
Apple GPUs
Vulkan
Cross-vendor GPUs
OpenCL
Cross-platform GPUs & CPUs
SYCL
Cross-platform C++ SYCL
WebGPU
Web & Native (GPU in browser)
Other Backends
(ROCm, oneAPI, Custom)
KEY BENEFITS
High Performance
True hybrid search at scale
Structure-Aware Search
Field / path aware semantic search
Runs Everywhere
From laptops to servers & edge
Local-First
Privacy, security & data sovereignty
Open Source
FLOSS (Apache 2.0)
No vendor lock-in
LEGEND
Control / Query Flow
Retrieval & Indexing Flow
Model / Inference Flow
Vector / ANN Flow

Vector search performance

Measured on the M1Pro (so the search time itself is marginal)

System	Approx QPS (768d)	Notes
Qdrant	~1k–3k	depends heavily on HNSW tuning
Milvus	~2k–5k	heavier infra
FAISS	~5k–10k	optimized C++ but limited system features
Elasticsearch	usually <2k	heavy overhead
Quarry HNSW	13k	C++.

Choose the objective before choosing k

Prediction quality · Benchmark comparability · Retrieval usefulness

01 CLASSIFICATION

Classification kNN

k controls voting and smoothing

CANDIDATE NEIGHBORHOODS

k=1 k=3 k=5 k=9 k=15

Cross-validation → validated k*

HOW TO CHOOSE

Test candidate k values on labeled data.
Compare held-out predictive performance.

CRITERION

Prediction loss / accuracy

Balance noise sensitivity and smoothing.

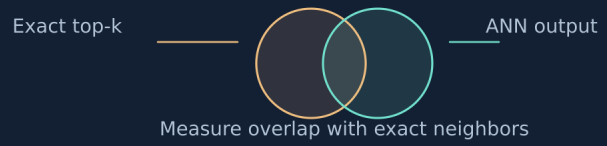
The chosen k can be fixed after validation.

02 CONTROLLED EXPERIMENT

ANN benchmarking

k defines a common retrieval task

Return k = 10 neighbors



HOW TO CHOOSE

Hold k fixed for a benchmark workload.
Compare search configurations fairly.

CRITERION

Neighbor recall ↔ throughput

Index memory and build cost also matter.

Agreement with vectors measures ANN fidelity.

03 APPLICATION OBJECTIVE

Production IR

k sets a candidate / evidence budget

QUERY-DEPENDENT EVIDENCE

Query A

Query B

Validate fixed k or a variable-size selection policy.

HOW TO CHOOSE

Evaluate the policy on judged queries.
Account for the corpus, task and budget.

CRITERION

Relevance · Coverage · Cost

Tune for useful and sufficient evidence.

Judge usefulness against the information need.

SEPARATE CONTROLS

k result count / output budget

`ef_search` HNSW search breadth

Fixed k helps standardize the task.
It does not fix all compute, memory or token costs.

CoreQuarry: kNN · Radius · Relative · Epsilon · Adaptive · Auto-tuning

Choose the retrieval policy against the user's task.

Schemas and counts are illustrative. Classification tuning, ANN workload control and IR policy selection are distinct choices.
Some vector databases already offer score/distance thresholds and automatic cutoffs; this is a comparison of objectives.

Sources: scikit-learn · ANN-Benchmarks · hnswlib · Weaviate

Adaptive neighborhoods: a better fit for IR

Useful result counts vary with the query. Let the neighborhood guide selection.

ONE HNSW INDEX · MULTIPLE RETRIEVAL POLICIES

Useful match · Weak match admitted · Useful match omitted · Other candidate

	Fixed kNN (k = 5)	Adaptive HNSW (QUERY-AWARE)
FEW STRONG MATCHES A small useful neighborhood.	 **5 returned** 2 useful + 3 weak The quota reaches beyond the strong matches.	 **2 returned** Keep the strong pair A tighter neighborhood limits weak additions.
MANY STRONG MATCHES A larger useful neighborhood.	 **5 returned** 4 useful matches omitted The same quota cuts through a useful cluster.	 **9 returned** Cover the useful cluster A broader neighborhood retains more evidence.

COREQUARRY RETRIEVAL CONTROLS

kNN · k	Radius · r	Relative	Epsilon · ε	Adaptive	Auto-tuning
Fixed result count	Distance-bounded results	Relative selection criteria	Epsilon-based control	Query-dependent selection	Automatic parameter tuning

Adapt the neighborhood to the query.

Fewer weak additions · Fewer useful omissions · Query-dependent result counts

Conceptual illustration; counts are illustrative. Benefit depends on the embeddings, corpus and tuning. HNSW remains approximate.
Adaptation of result selection and adaptation of HNSW search effort are separate controls.

Research context: Kratzwald & Feuerriegel, EMNLP 2018

Bare Metal, Sovereign, COTS Hardware (The hardware you have sitting in front of of you)

The benchmarks are mainly on this machine— the one I'm presenting on:

Apple M1Pro, 16GB RAM, external NVME disk (WD SN750 2TB) using APFS and connected with Thunderbolt 3.

The M1Pro has 200 GB/s memory bandwidth. Max SSD (even another disk) 2,500 – 2,800 MB/s.

M5 Max (40-Core GPU Variant): Features a unified memory bandwidth of **614 GB/s and it can get as high as** 6,000 to 6,500 MB/s SSD throughput.

Threadripper PRO 9000 WX-Series: 409 GB/s and SSD as high as 14,000 to 14,900 MB/s.
The new MS Surface (GB10) is 300 GB/S but has ~14,500 MB/s disk I/O.

SCALE · PATENTS

Two billion words, one pass, one laptop.

656,519	2.02 B	48 min	6.8 GB
records	words	2,861 s to index	peak resident, no swap

M1 Pro, 16 GB, 2021. Source on an external NVMe disk over Thunderbolt 3.

SCALE · PATENTS

Two words, 656,519 patents, one claim.

```
$
lsearch -d PATENTS -show -max 1 risc matrix PEER
1 record of 656519 matched your query
Evidence: [CLAIM-TEXT]
ors, including said RISC
processor and said transfer processor, said m memory
sources and said switch matrix, including said lin
```

49 ms

PEER: the engine finds the field

20–27 ms

AND:CLAIM-TEXT: you name it. Same claim.

PEER asks for both words in the same field and names it. If you already know the field, saying so is about twice as fast.

SCALE · WIKIPEDIA (2 million record ES Bulk Dump)

The second query is the one that matters.

editors

6 ms

300 hits, capped: the word is everywhere

defaultsort/editors

4 ms

one record: the field that was asked for

An agent that sees a flood can narrow with one more query, and each query costs milliseconds.

DISPLAY EVIDENCE

Show the passage that fits the reader, not the record.

```
$
lsearch -d /tmp/P -show -max 2 patent
2 records of 86755 matched · 24 ms
Evidence: [P]
graphic emulsions used in
such methods are described in U.S. Patent Nos. 2,456,953,

2,497,875, 2,497,876, 2,497,917, 2,507,154,
```

[P] is the paragraph the hit sits in. The record is 86,755 patents' worth of XML.

1 **Choose the evidence.** Covers and single hits compete; each extra byte past the budget costs more.

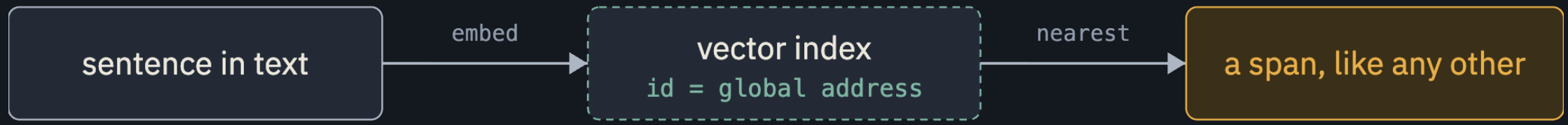
2 **Find where it lives.** One structural probe at a word inside it, then its ancestors.

3 **Fit the budget.** Whole container if it fits; otherwise pad around the evidence, but never past the container.

Structure is presentation, not evidence. It never competes with the words that matched.

SEMANTIC SEARCH

Semantic search is a view, not a second database.



Because the hit is an address, it can be intersected with fields, records and word matches.

SEMANTIC SEARCH

A single vector can’t return every combination of documents.

THE PROOF

The number of top-k sets a d-dimensional embedding can return is bounded by d. Some pairs of documents can never be returned together, for any query.

Weller, Boratko, Naim & Lee, *On the Theoretical Limitations of Embedding-Based Retrieval*, 2025 (ICLR 2026)

THE TEST: “WHO LIKES QUOKKAS?”

10.0
Gemini Embed, 3072-d, recall@100

18.9
Promptriever, 4096-d, recall@100

50,000 documents, 1,000 one-attribute queries. A cross-encoder solved all 1,000.

A vector is a lossy view. Keep the exact evidence beside it.

HYBRID EVIDENCE

Every kind of evidence becomes energy before it is combined.

WORDS

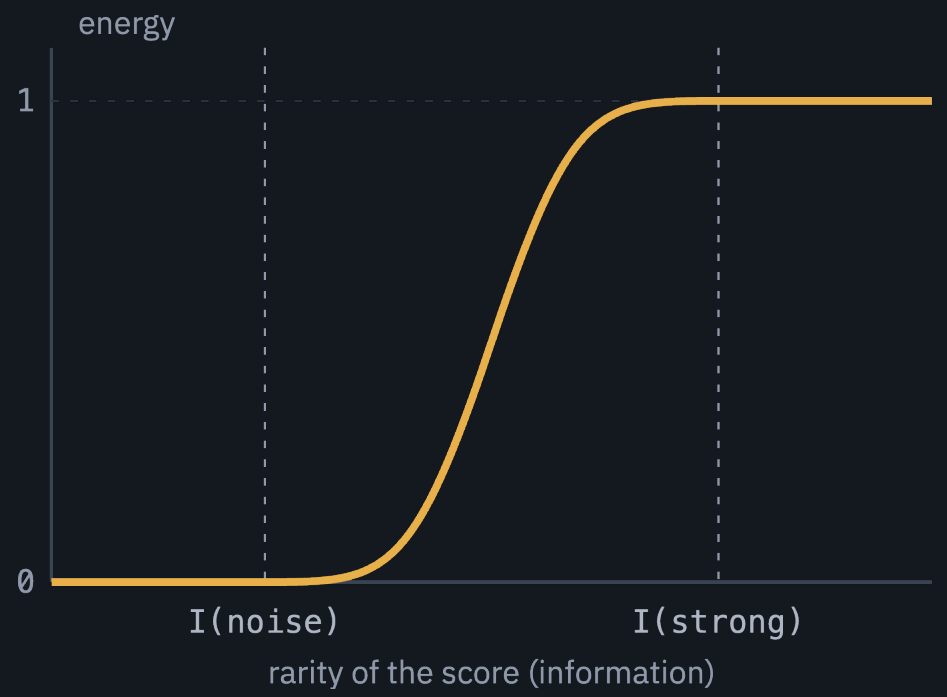
```
energy = found / possible
```

Two of three query terms inside the span: energy 0.67. A fraction of the evidence the query asked for.

VECTORS

```
tail = (N – rank + 1) / (N + 1)
information = –log(tail)
```

A cosine score means nothing alone. How rare is it against a sampled background for this model and field?



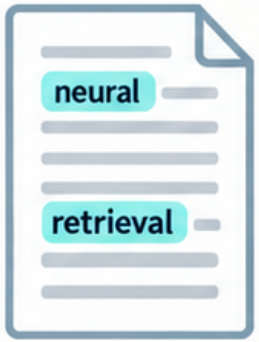
Rarity is not relevance. It only gives vectors and words a common, bounded scale.

IB: Lexical + vector, one score

Normalize lexical scores • Calibrate vector evidence • Add weighted contributions

Lexical scores

Default TF–IDF / L2 path



$$r_d = tf_d \times idf$$

$$L_d = \frac{r_d}{\sqrt{\sum_j r_j^2}}$$

j: matches for this lexical operand

Other lexical normalizers are selectable.

Vector evidence

Schmate cosine/IP score

$$s = \frac{1 + \text{similarity}}{2}$$

$$x = \text{clamp}\left(\frac{s-a}{b-a}, 0, 1\right)$$

$$E_v = f + (1-f)\,x^2(3-2x)$$

a	VectorNoise = 0.5
b	VectorStrong = 1.0
f	VectorFloor = 0.0

Fixed anchors; independent of top-K

Weighted score algebra

$$S_d = w_L L_d + w_V E_v(d)$$

Weights: per-operand TermWeight
AND / OR determine membership

One ranked result set

Hit coordinates retained

No RRF stage • Reranker optional

Worked example

Illustrative weights: $w_L = 1,\ w_V = 0.5$

Lexical: $r = (3, 4) \rightarrow L = (0.60, 0.80)$

Vector A: similarity $0.60 \rightarrow s = 0.80 \rightarrow E_V = 0.648$

Record	Lexical L	Vector E_v	Combined S
A	0.600	0.648	**0.924**
B	0.800	0.000	0.800

Optional: background calibration

Model / field CDF → Surprisal $I(s)$ → Same smoothstep

$$x = \text{clamp}\left(\frac{I(s) - I(a)}{I(b) - I(a)}, 0, 1\right)$$

Calibrate against background, not current top-K.

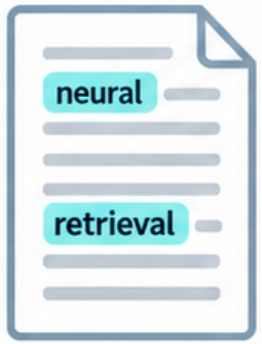
Tune anchors and weights per model / field • Normalized scores are not relevance probabilities

IB: Lexical + vector, one score

Normalize lexical scores • Calibrate vector evidence • Add weighted contributions

Lexical scores

Default TF–IDF / L2 path



$$r_d = tf_d \times idf$$

$$L_d = \frac{r_d}{\sqrt{\sum_j r_j^2}}$$

j: matches for this lexical operand

Other lexical normalizers are selectable.

Vector evidence

Schmate cosine/IP score

$$s = \frac{1 + \text{similarity}}{2}$$

$$x = \text{clamp}\left(\frac{s-a}{b-a}, 0, 1\right)$$

$$E_v = f + (1-f)\,x^2(3-2x)$$

a	VectorNoise = 0.5
b	VectorStrong = 1.0
f	VectorFloor = 0.0

Fixed anchors; independent of top-K

Weighted score algebra

$$S_d = w_L L_d + w_V E_v(d)$$

Weights: per-operand TermWeight
AND / OR determine membership

One ranked result set

Hit coordinates retained

No RRF stage • Reranker optional

Worked example

Illustrative weights: $w_L = 1$, $w_V = 0.5$

Lexical: $r = (3, 4) \rightarrow L = (0.60, 0.80)$

Vector A: similarity $0.60 \rightarrow s = 0.80 \rightarrow E_v = 0.648$

Record	Lexical L	Vector E_v	Combined S
A	0.600	0.648	**0.924**
B	0.800	0.000	0.800

Optional: background calibration

Model / field CDF → Surprisal $I(s)$ → Same smoothstep

$$x = \text{clamp}\left(\frac{I(s) - I(a)}{I(b) - I(a)}, 0, 1\right)$$

Calibrate against background, not current top-K.

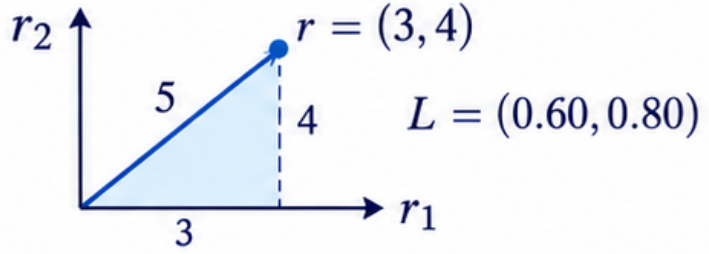
Tune anchors and weights per model / field • Normalized scores are not relevance probabilities

IB: E2 normalization in two phases

L2 scaling in score space → coverage and coherence in document space

Phase 1 Euclidean score scaling

TF–IDF / L2, per lexical operand



$$r_{td} = tf_{td} \times idf_t$$

$$L_{td} = \frac{r_{td}}{\sqrt{\sum_j r_{tj}^2}}$$

$$S_{base}(d) = \sum_t w_t L_{td}$$

j: matching records • t: lexical operands

Phase 2 Coverage × positional coherence

Coverage (distinct lexical sources)

$$c = \frac{\text{AuxCount}}{\text{MaxAuxCount}}$$

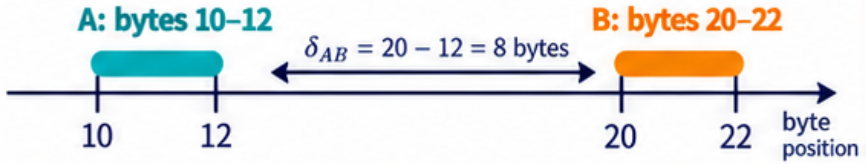
$$W_{cov} = f + (1 - f)c$$

f = CoverageFloor = 0.5 [E2]

MaxAuxCount: maximum in the current result set

Positional coherence (document space)

Nearest interval gap per distinct lexical source pair



$$g(\delta) = 1 + \frac{3}{1 + \delta/8} \qquad K = \sqrt{GM[g(\delta_{ab})] \times g(\delta_{max})}$$

$$W_{prox} = 1 + (K - 1)c$$

GM: geometric mean across distinct source pairs

δ_{max}: largest nearest-pair gap

$K = 1$ when no eligible pair exists

8-byte scale in ranking.c • Same-source and anonymous pairs skipped

$$S_{E2} = S_{base} \times W_{cov} \times W_{prox}$$

Coverage gates the positional bonus.

Controlled comparison: $S_{base} = 0.60$ for all rows; MaxAuxCount = 2

Evidenc e	Coverage c	Nearest gap	K	E2 score
Close + complete	1.00	8 bytes	2.5000	**1.5000**
Wide + complete	1.00	80 bytes	1.2727	0.7636
Partial: one source	0.50	No pair	1.0000	0.4500

Current implementation • Document distances use byte coordinates • E2 scores may exceed 1

HYBRID EVIDENCE

A cover wins by holding more evidence in less space.

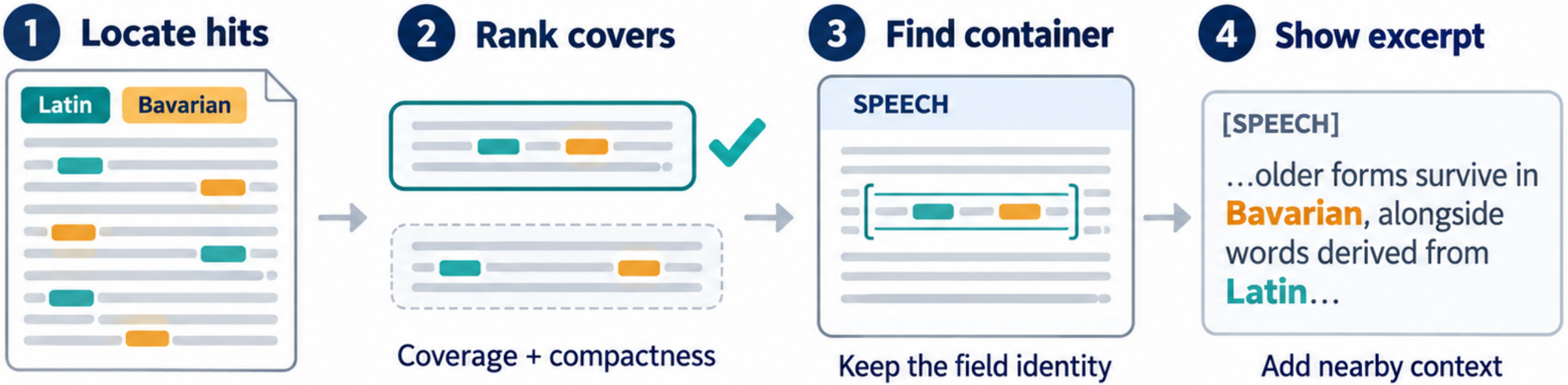
rank = energy
× 1 / (1 + dispersion / 32)
dispersion = extent width in bytes ÷ evidence items

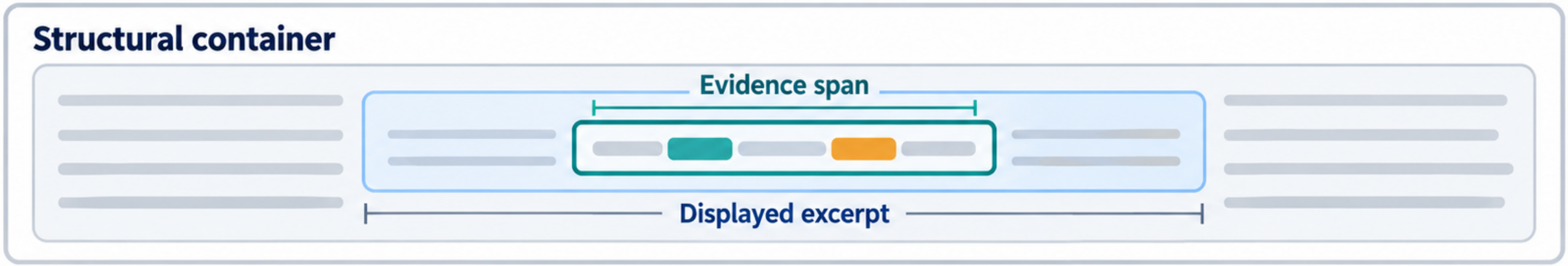
0.62	0.28	0.14
both terms within 40 bytes	one term, 8 bytes	both terms spread over 400 bytes

Worked example for a two-term query, computed from the formula. Illustration, not a measurement.

IB: From query hits to evidence

Coverage + compactness + document structure





Byte advice is a soft budget; selected evidence stays intact.

SEMANTIC SEARCH

Similar is not the same as true.

query

who won the 2016 election

```
default32]> search who won the 2016 election
 - [score=0.897411, sid=9, label=8, tokens=[0,9]] Donald Trump won the election for President against Clinton
 - [score=0.884343, sid=10, label=9, tokens=[0,6]] Biden won the election against Trump
 - [score=0.880801, sid=8, label=7, tokens=[0,8]] Hilary Clinton lost the election against Donald Trump
 - [score=0.876038, sid=491, label=490, tokens=[0,23]] The 2016 presidential election shocked political observers
when Donald Trump defeated Hillary Clinton despite losing the popular vote by nearly three million ballots.
 - [score=0.871397, sid=507, label=506, tokens=[0,27]] Joe Biden defeated Donald Trump in the 2020 presidential
election with over seven million more votes and 306 electoral votes in a contest marked by record turnout.
[default32]>
```

A vector finds text about the question. It does not know which text answers it.

SEMANTIC SEARCH

Similar is not the same as true.

who won the 2016 election

0.897	Donald Trump won the election for President against Clinton
0.884	Biden won the election against Trump
0.881	Hilary Clinton lost the election against Donald Trump
0.876	The 2016 presidential election shocked political observers when Donald Trump defeated Hillary Clinton…
0.871	Joe Biden defeated Donald Trump in the 2020 presidential election…

Rank 2 gets the winner wrong, and the only sentence that says “2016” ranks fourth.

VECTOR SCALE

In high dimensions, some vectors are everyone's neighbour.

HUBNESS

"Popular" nearest neighbors: a few points appear in far more neighbour lists than the rest, and the skew grows with dimension.

Radovanović, Nanopoulos & Ivanović, *Hubs in Space*, JMLR 2010

HNSW RUNS ON THEM

Hubs act as highways that route queries early in the search. The hierarchy was supposed to do that job.

$d < 32$ is where the hierarchy helped. Our embeddings are 384 to 4096.

Munyampirwa, Lakshman & Coleman, *The 'H' in HNSW Stands for "Hubs"*, ICML 2025 workshop

The thing that makes the graph fast is the same thing that makes some vectors show up for everything.

VECTOR SCALE

Sharding bounds the graph. It doesn’t spread the answer.

WHY SHARD

183 GB

peak memory to build a 100M-vector HNSW graph (BigANN, hnswlib)

WHY IT ISN’T ENOUGH

Each shard returns its own top-k. Here k = 5: shard 2 holds seven relevant vectors and loses two, while three shards return five non-answers each.

VECTOR SCALE

Keep shards small: about 50,000 vectors.

50k	100k	>100k
comfortable	still OK, the ceiling	quality and latency fall off in our tests

[Your measurements here: recall@10 and latency at 50k, 100k and 200k vectors, same model, same machine]

1,000,000 vectors is 20 shards. A rule like that only works if you can say which shards to ask.

VECTOR SCALE

Context anchors a vector. It doesn’t solve scale.

WHAT OTHERS PUBLISH

Milvus, manual partitions	~20–100K rows
Weaviate, flat → HNSW	10,000 objects
Weaviate, flat-search cutoff	40,000
Weaviate, before enabling PQ	10K–100K per shard
Qdrant, binary quantization	up to 32× smaller

OUR RULE OF THUMB

50k–100k

vectors per shard, from our own tests on commodity hardware

Embeddings crowd into a narrow cone (anisotropy). Cookie-cutter text like abstracts should crowd them further, so the one sentence that carries the finding hides among look-alikes. Our own top-5 scores for one query span 0.897 to 0.871.

VECTOR SCALE

IVF scales. It also needs a plan.

Train first

Cluster centres have to be learned from the data before anything can be added.

Uneven cells

Lists end up very different sizes, so query cost depends on which cells a query lands in.

Tuned probing

Recall depends on how many cells you visit. Neighbours across a boundary are missed unless you probe more.

Drift

When the collection changes, the centres go stale and the index has to be rebuilt.

That suits a managed cluster. It is harder to ask of a laptop that appends documents all day.

BACK TO THE START

Exact search has no hubs.

A word hit is a fact about the text. A typed field, a date or a number is exact. Neither has a popularity bias or a capacity limit.

So use them to say where to look, and let the vector say what is similar inside it.

risc matrix AND:CLAIM-TEXT

656,519 → 1

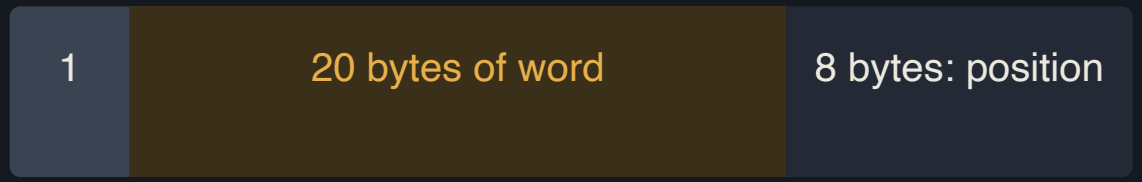
patents, narrowed to the claim in about 17 ms

The vector hit is a span too, so the two intersect without a second database.

FOOTPRINT

A 198 MB table in front of two billion words.

ONE ENTRY, 29 BYTES



Fixed stride, sorted by unsigned byte comparison. A word longer than 20 bytes is marked with * and only its posting range goes back to the text.

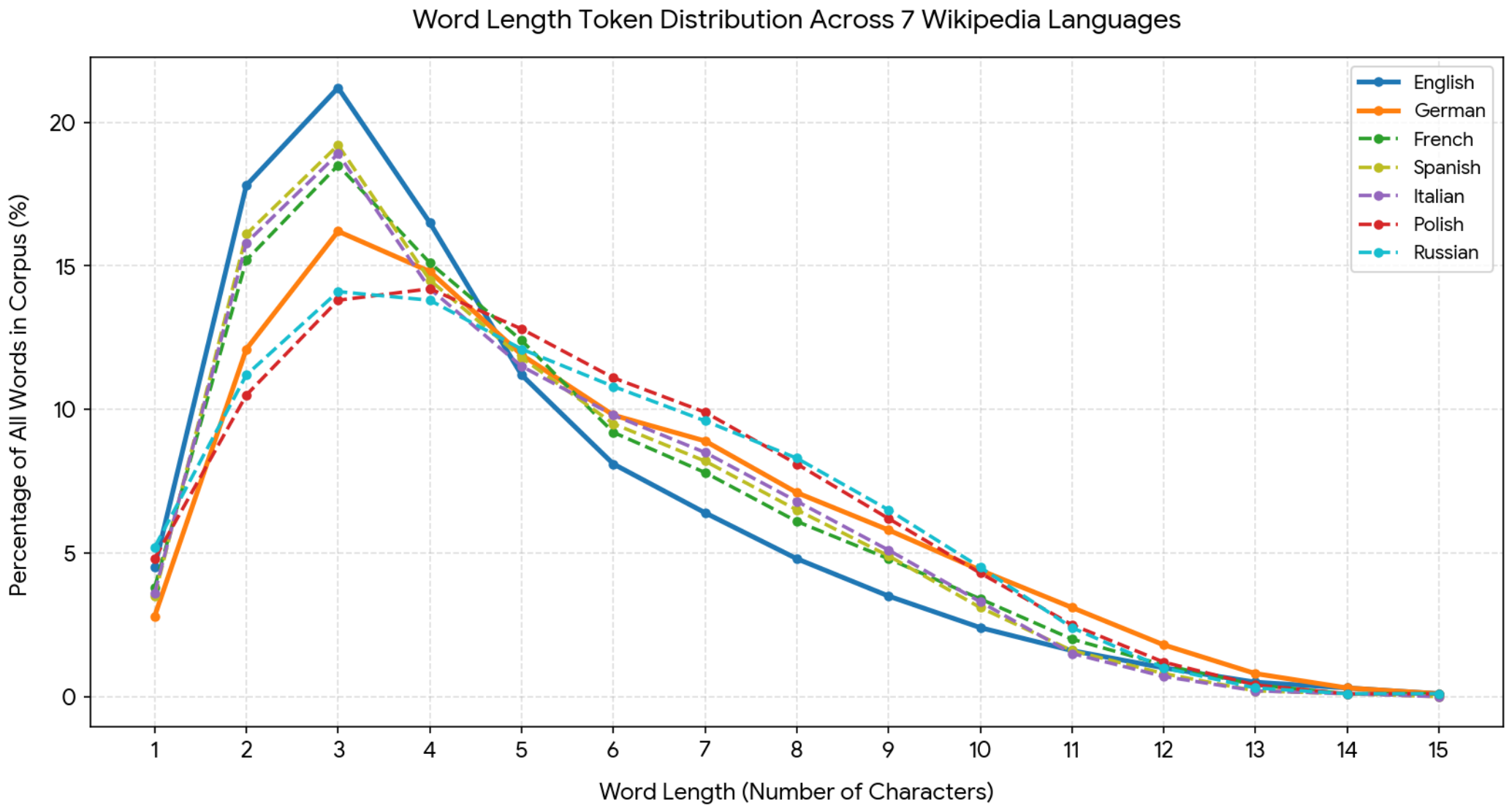
6,817,829

entries in the patents table, about 197.7 MB

7.9%

of entries overflow (538,047)

Tiered Index Cache (Why it works):



Many processes, shared index pages

mmap, bounded caches and continuous append put the OS memory system to work.

CONCURRENT SEARCH SESSIONS · ONE HOST / ONE KERNEL

Search process A

PRIVATE WORKING STATE

Query state · result sets · page tables

HNSW / mapping LRU · bounded

VIRTUAL FILE MAPPINGS

page 7 | page 8 | page 9

Search process B

PRIVATE WORKING STATE

Query state · result sets · page tables

HNSW / mapping LRU · bounded

VIRTUAL FILE MAPPINGS

page 7 | page 8 | page 9

Search process C

PRIVATE WORKING STATE

Query state · result sets · page tables

HNSW / mapping LRU · bounded

VIRTUAL FILE MAPPINGS

page 7 | page 8 | page 9

Indexer / appender

Documents keep arriving

Parse and index

Append data / write shard(s)

Publish completed data

Searches continue on published data.

Separate virtual views → same files and offsets

Append / publish

OS FILE CACHE / SHARED PHYSICAL RAM

page 7 — one resident copy

page 8 — one resident copy

page 9 — one resident copy

The reader mappings can reference these same physical pages.

Warm reads reuse RAM.

Cold pages arrive on demand.

The OS can reclaim cached pages.

Two cache layers

Application LRU

Retains recently used objects / mappings

OS file cache

Controls which file pages are resident

An mmap mapping does not pin all its pages.

Cache miss → demand read / page-in

FILESYSTEM / INDEX STORAGE

Shard 1 | Shard 2 | Field tables | Vector backing data

Published index files on storage · accessed through the OS file cache

Appended data / new shards

New searches / refresh reopen or remap.

$$\text{Search RAM} \approx \text{shared resident file pages} + \Sigma(\text{private working sets})$$

Additional searches add private state and CPU work; hot file pages can be reused.

Tune I/O layout, readahead and cache budgets for the workload. Journaling is a separate durability trade-off.

Sharing shown within one host/kernel for the same backing files and offsets. Read-only index mappings shown; new extents need refresh/remapping.

References: Linux kernel page-cache and ext4 documentation · mmap(2) · Apple virtual-memory documentation

FOOTPRINT

Develop on the floor, not the ceiling.

EMBEDDING ONE QUERY, M1 PRO (METAL)

MiniLM-L12, f32	4 ms · 15 ms
bge-large, q8_0	15 ms · 35 ms

First figure: 11 tokens. Second: 137 tokens.

THE FLOOR

A 2021 laptop built the patents index with no swapping. If it works there, a bigger machine is a bonus, not a requirement.

[Other machines we benchmark: M3 Pro, M4 Pro, AGX Orin, RTX 50 laptop, 2× GTX 1080]

Vectorization performance

```
Loading /Users/edz/Downloads/bge-large-en-v1.5-q8_0.gguf on accelerated backend (threads=6)...
Model: bge-large-en-v1.5  arch=bert  n_embd=1024  load=220 ms
Warmup=5 iters, measure=50 iters

== latency by input length (batch=1) ==
 very-short    tok= 11  bs=  1  | mean=  14.82 ms  med=  14.82  p95=  14.94  min= 14.60  max= 15.05  |    742 tok/s    67.5 seq/s
 short         tok= 38  bs=  1  | mean=  20.79 ms  med=  20.71  p95=  20.98  min= 20.47  max= 22.99  |   1828 tok/s    48.1 seq/s
 medium        tok=137  bs=  1  | mean=  38.63 ms  med=  38.62  p95=  38.74  min= 38.54  max= 38.86  |   3546 tok/s    25.9 seq/s
 long          tok=263  bs=  1  | mean=  67.24 ms  med=  67.21  p95=  67.38  min= 67.12  max= 67.55  |   3911 tok/s    14.9 seq/s
 max           tok=512  bs=  1  | mean= 116.45 ms  med= 116.62  p95= 117.12  min=115.65  max=117.53  |   4397 tok/s     8.6 seq/s

== throughput by batch size (tok=128) ==
 bs=1          tok=137  bs=  1  | mean=  38.63 ms  med=  38.61  p95=  38.80  min= 38.49  max= 39.08  |   3546 tok/s    25.9 seq/s
 bs=4          tok=137  bs=  4  | mean= 135.44 ms  med= 135.41  p95= 135.74  min=135.21  max=136.05  |   4046 tok/s    29.5 seq/s
 bs=8          tok=137  bs=  8  | mean= 266.41 ms  med= 266.36  p95= 266.81  min=266.07  max=266.91  |   4114 tok/s    30.0 seq/s
 bs=16         tok=137  bs= 16  | mean= 530.99 ms  med= 529.86  p95= 536.17  min=529.20  max=539.27  |   4128 tok/s    30.1 seq/s
 bs=32         tok=137  bs= 32  | mean=1069.06 ms  med=1067.57  p95=1074.02  min=1066.46  max=1085.44  |   4101 tok/s    29.9 seq/s
```

Performance on embedded hardware (Orin64)

```
Loading /Users/edz/Downloads/bge-large-en-v1.5-q8_0.gguf on accelerated backend (threads=6)...
CUDA Device 0: Orin (65.85 GB VRAM)
bert: using CUDA backend (full get_rows support)
Model: bge-large-en-v1.5  arch=bert  n_embd=1024  load=1014 ms
Warmup=5 iters, measure=50 iters

== latency by input length (batch=1) ==
 very-short    tok= 11  bs= 1  | mean=  9.97 ms  med=  8.59  p95=  14.95  min=  7.27  max= 20.27  |    1103 tok/s    100.3 seq/s
 short         tok= 38  bs= 1  | mean=  9.72 ms  med=  9.81  p95=  10.02  min=  9.21  max= 10.31  |    3911 tok/s    102.9 seq/s
 medium        tok=137  bs= 1  | mean= 16.37 ms  med= 16.33  p95=  16.53  min= 15.66  max= 17.54  |    8368 tok/s     61.1 seq/s
 long          tok=263  bs= 1  | mean= 27.97 ms  med= 28.00  p95=  28.27  min= 27.18  max= 29.45  |    9402 tok/s     35.8 seq/s
 max           tok=512  bs= 1  | mean= 46.53 ms  med= 46.43  p95=  47.57  min= 45.86  max= 48.14  |   11004 tok/s     21.5 seq/s

== throughput by batch size (tok=128) ==
 bs=1          tok=137  bs= 1  | mean= 16.16 ms  med= 16.24  p95=  16.64  min= 15.59  max= 18.18  |    8477 tok/s     61.9 seq/s
 bs=4          tok=137  bs= 4  | mean= 55.13 ms  med= 55.10  p95=  55.39  min= 54.70  max= 56.46  |    9941 tok/s     72.6 seq/s
 bs=8          tok=137  bs= 8  | mean=113.15 ms  med=112.92  p95= 114.58  min=112.46  max=115.63  |    9686 tok/s     70.7 seq/s
 bs=16         tok=137  bs=16  | mean=215.03 ms  med=214.71  p95= 216.90  min=214.31  max=218.90  |   10194 tok/s     74.4 seq/s
 bs=32         tok=137  bs=32  | mean=427.06 ms  med=427.35  p95= 428.06  min=425.14  max=428.74  |   10265 tok/s     74.9 seq/s
```

FOOTPRINT

Indexing Performance (Example).

~700,000

words per second, 2 million words. 656519 Patents

1,459 s

Wikipedia, 753 million words. 90666 articles

0

swaps on the runs

Global counters assign every address as the text streams past, then sorted runs are merged. No cleaning, no schema, no second pass.

HONEST LIMITS

What it will not do, and where it stops.

HARD CEILINGS

16 million records per physical index (24-bit record id). [Max 160 HNSW shards if 100k 1:record]

255 indexes per virtual database (8-bit slot).

Addresses are 64-bit across all of it.

NOT CLAIMED

A relevance score on a public benchmark.

Evidence ranking is a heuristic, separate from structure.

Case folding is complete for common scripts, not all of Unicode,

(*) To keep some perspective: the SEC has only 1.2-1.7 million fillings per year (~4500/day) The USPTO gets around 600,000 formal patent applications each year. The NYT publishes about 50-70k pieces of journalism each year.

Main Character Sets not yet supported

Samaritan
Mandaic
Syriac Supplement
Devanagari Extended
Devanagari
Bengali
Gurmukhi Gujarati
Oriya
Tamil
Telugu
Kannada
Malayalam
Sinhala
Thai
Lao
Tibetan
Myanmar

Georgian (old range)
Hangul Jamo
Ethiopic
Ethiopic Supplement
CherokeeUnified Canadian Aboriginal Syllabics
Ogham
Runic
Tagalog / Hanunoo / Buhid / Tagbanwa
Khmer
Mongolian
Unified Canadian Aboriginal Syllabics Ext
Limbu

AI Security/Governance (FOSDEM ’24)

https://archive.fosdem.org/2024/schedule/event/fosdem-2024-2079-fortify-ai-against-regulation-litigation-and-lobotomies/

“Fortify AI against regulation, litigation and lobotomies”

AI models, particularly LLMs, readily and sometimes unwittingly may plagiarize content—what constitutes acceptable use is currently not fully resolved— opening their users to copyright tort and other forms of litigation. For society an immanent danger is also in how easily they can be poisoned or lobotomized to propagate disinformation. It is also difficult to uncover and address sources of bias. The later is of particular legislative concern—see emerging regulations such as the EU’s AI Act as a prototype for whats to come. Without traceability for data with clear license expressions and provenance, AI models are easy targets.

AN OPEN ISSUE

Prompt injection is a retrieval problem too.

CHUNK RAG

Any text that matches goes into the prompt, whoever wrote it, whichever part of the page it came from.

SPANS

Every passage has an address and a field. Ask only for the claim text, or only for one author’s lines, and nothing else is fetched.

It narrows what can reach the model. It does not make text safe.

The ib search algebra

Sets, positions, document structure and relevance — composed in one query.

CLOSED UNDER COMPOSITION

$R_1 \otimes R_2 \rightarrow R_3$

Every result can become the next operand.

Record identities • Relevance scores • Source hit spans

8 FAMILIES

01 Boolean sets

Combine or exclude matching records

```
AND OR NOT
ANDNOT NOTAND
XOR XNOR
NAND NOR
```

02 Position & order

Constrain source distance and direction

```
ADJ NEAR[:n] FAR
PROX:n / PROXIMITY:n
BEFORE[:n] AFTER[:n]
PRECEDES FOLLOWS
NEIGHBOR DIST<op>n
```

03 Document structure

Relate hits through field containers

```
PEER PEERb PEERa
PEER:n PEERb:n PEERa:n
XPEER
SIBLING
```

SIBLING rewrites compatible expressions

04 Named fields

Scope hits or require a shared instance

```
WITHIN:f INSIDE:f
XWITHIN:f INCLUSIVE:f
NOT:f AND:f OR:f
BEFORE:f AFTER:f
```

05 Source & record filters

Select identity, format, source and date

```
WITHKEY:p KEY:p
FILE:p
EXTENSION:e DOCTYPE:t
WITHIN:date-range
```

06 Adaptive matching & score

Choose fallback sets and adjust relevance

```
MAYBE NARROW
PROMOTE DEMOTE
BOOST:w
```

Fallback set choice • Score influence

07 Result controls

Retain, focus, count, trim and order

```
REDUCE:n FOCUS:n
HITCOUNT:n
HITCOUNT<op>n
TRIM:n SORTBY:criterion
```

08 Cross-index joins

Match record keys across physical indexes

```
JOIN JOINL JOINR
```

Index A `key = K` — Index B `key = K`

Key-based result-set composition

Directional join variants

COMPOSE THE QUESTION

```
virus vaccine PEER:130 WITHIN:CLAIM-TEXT BOOST:1.5 SORTBY:Score
```

Find peers within 130 bytes, restrict their evidence to claim text, apply a score weight, then sort.

n = distance / count by operator; f = field; p = pattern; e = extension; t = doctype; w = weight; <op> = <, ≤, >, ≥.
Distance bounds use source bytes; NEAR and PROX also accept relative distances. Aliases are shown selectively.

DIVISION OF LABOUR

The engine locates. The model understands.

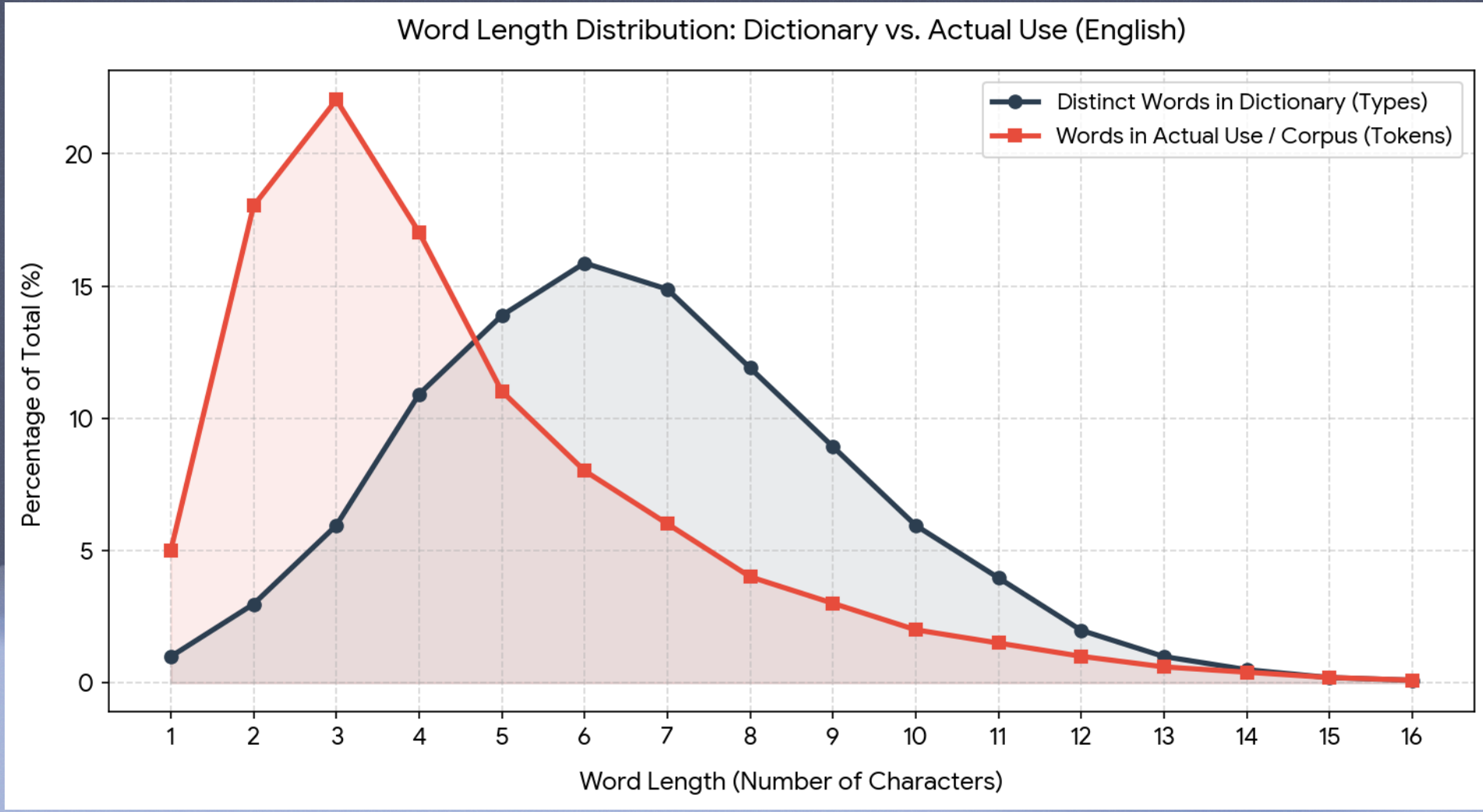
A search and retrieval engine for humans and agents, built to run on the hardware you already own.

NGI ZERO COMMONS

Architectural & Competitive

Competitor Tier / Paradigm	Representative Systems	Architectural Tax	Language / Runtime Issues	CoreQuarry's Advantage
Classical Inverted IR	Apache Lucene, Elasticsearch, Solr, Tantivy	Immense RAM burn for memory-resident FST term dictionaries; I/O-heavy background segment merges.	Java JVM fragmentation and unpredictable Garbage Collection stalls (Lucene/Solr).	**PatArray Architecture:** Elimination of traditional inverted dictionary bloat via cache-contiguous, virtual memory-mapped pointer tracking layouts.
Hybrid & Vector First DBs	Weaviate, Qdrant, Milvus	Vectors-first focus leaves them with weak lexical capabilities; require separate post-filtering cycles or cross-index stitching.	Go runtime memory isolation lag; massive multi-node infrastructure dependencies (Kubernetes).	**Hardware-Fused Co-Execution:** Unifies hard lexical masks and soft tensor weights natively inside single-pass graph traversal runs.
Specialized Vector Backends	FAISS, ScaNN, Baseline HNSWlib	No native transactional CRUD (FAISS/ScaNN); Baseline HNSWlib forces unquantized float32 bloat that exhausts CPU L3 cache pipelines.	Monolithic build matrices (ScaNN/Bazel); lack of native metadata layer capabilities.	**Turbo-Quantization Primitives:** Blistering, explicitly unrolled ARM Neon/SVE2 kernels driven by compact, low-bit block quantization layouts.
Embedded Databases	SQLite (with extensions), DuckDB	Row-store overhead/serialization penalties (SQLite); analytical column matrices entirely lack structural proximity and linguistic IR paths (DuckDB).	C++ native but bound to isolated virtual table bridges or transient batch-ingestion models.	**In-Process Fusion:** Zero serialization delays. Text boundaries, sequence configurations, and tensor spaces execute concurrently inside the host app process.
RAG Orchestration	LlamaIndex, LangChain	The "Abstraction Tax." Massive performance degradation by shifting core document segmentation and data mapping to Python object layers.	Python/TypeScript runtime speed ceilings; rely completely on network I/O or external cluster storage engines.	**Bare-Metal Efficiency:** Fuses the entire ingestion, parsing, tensor quantization, and hybrid discovery lifecycle into a single native runtime.
Structural Lexical Search	MarkLogic	Heavy enterprise server monolith; massive memory-resident cache footprints (List/Range Caches).	Proprietary closed-source distribution; completely un-optimized for embedded edge execution topologies.	**Gonnet/Baeza-Yates Inspired Lineage:** Elevates early structured text principles to modern silicon. Uses branchless SIMD and MSD Radix sorting over fixed-width 64-character compilation horizons.

CoreQuarry Lexical Index:



CoreQuarry Lexical Index: